UNIVERSITY OF PENNSYLVANIA | CIS 5450

Understand your NYC trip-data project

Complete cell code with explanations
from raw files to fare-prediction models

102 cells, in notebook order
Full original source, explanations of the operations, inputs and outputs, and connections between the data preparation, enrichment, plots, and model draft.

Prepared for Alex
October 22, 2026

Source: CIS5450_GroupProject_BeginningFile.ipynb

This document is a code companion. Each nonempty cell is reproduced before its explanation, including the original comments, inactive code, and existing mistakes. The explanations show how individual statements contribute to the overall project.

Code fidelity. Source blocks come directly from the uploaded notebook. Line numbers are added for reference; long lines wrap visually and tabs are expanded for display. The original notebook has not been edited. Saved outputs were inspected, but the full pipeline was not rerun.

Reading map

Start with the project overview and data-flow diagram. The main walkthrough then presents each original cell, explains its behavior, and identifies how it depends on earlier work. PDF bookmarks jump to any phase or individual cell. The final sections explain the disconnected parts of the current workflow and the model sequence those parts are intended to support.

> **How cell numbers work.** Cell 1 is the first cell in the file. Numbering includes Markdown and empty cells, for 102 total cells: 82 code cells and 20 Markdown cells. It is not the execution counter shown beside a Jupyter cell.

Reading map 2

What the project is trying to do 3

Data flow and variable ownership 4

Read the data correctly 5

Cells 1-10: Setup and downloads 6

Cells 11-22: Sampling and SQL views 12

Cells 23-32: Combining and cleaning trips 20

Cells 33-39: Weather enrichment 27

Cells 40-51: Geography and holidays 32

Cells 52-73: Legacy pandas workflow 36

Cells 74-80: Exploratory visualizations 44

Cells 81-102: Spark and model draft 48

Problems to resolve before trusting results 58

A coherent modeling workflow 59

Operations used throughout the code 60

Sources and review scope 61

What the project is trying to do

The notebook assembles a sample of NYC yellow taxi and high-volume rideshare trips, cleans and aligns the records, adds contextual data, and starts a regression workflow for predicting trip charges. The prediction goal is inferred from target = df_cleaned["total_charged"] and the selected regression models; the notebook does not yet define a precise prediction-time scenario.

> **Current state:** data acquisition and integration are substantially developed. The modeling portion is a draft. A fresh top-to-bottom run encounters environment and ordering problems, and the enriched features do not currently reach the models.

The unit of observation

One row generally represents one completed trip. Weather starts as a separate table with one row per time interval. Zone data is a lookup with one row per location identifier. The project joins or maps that context onto trip rows; it does not turn the table into a demand time series or pair equivalent taxi and rideshare journeys.

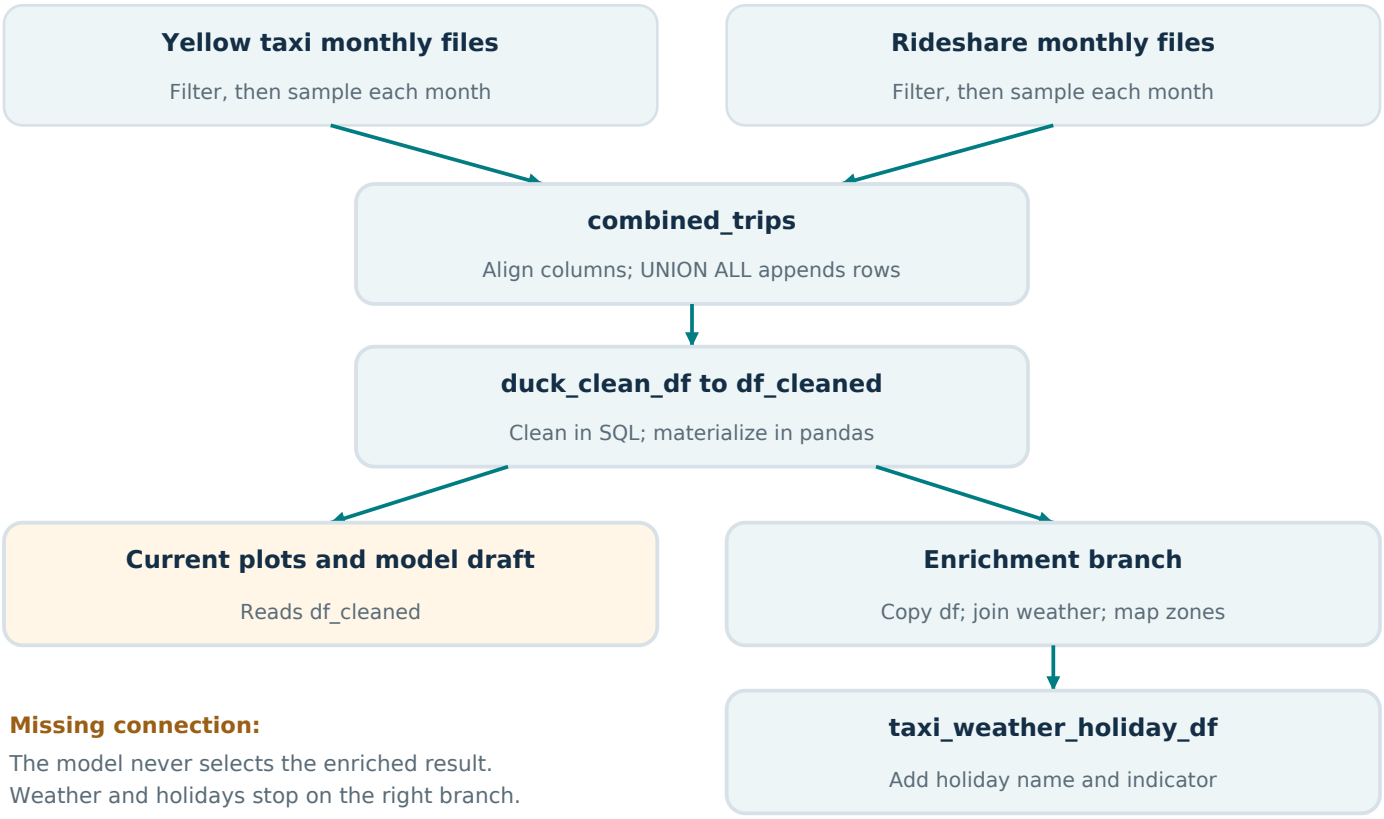
Stages and outputs

Stage	Operation	Result
Acquire	Choose months and download two services	Full monthly Parquet files
Reduce	Filter and sample 5,000 rows per monthly file	Smaller sample files
Integrate	Rename fields and append service records	combined_trips SQL view
Clean	Filter endpoint IDs and missing timestamps	duck_clean_df SQL view
Enrich	Join weather; map zones; label holidays	taxi_weather_holiday_df
Explore	Correlations, counts, and scatter plots	Descriptive figures
Model	Split, scale, attempt PCA and regressions	Incomplete model workflow

What a result would mean

A descriptive association can show that sampled trips with certain characteristics have different charges. It does not by itself prove that weather or holidays cause those differences. A predictive score is meaningful only after the target, valid inputs, evaluation split, and preprocessing are defined. Equal per-month samples also require care when interpreting population-wide demand or service shares.

Data flow and variable ownership



Missing connection:

The model never selects the enriched result.
Weather and holidays stop on the right branch.

Object	Type and role
con	DuckDB connection that owns the temporary views.
yellow_taxi / fhvhv	Views over selected sample files; yellow receives extra filters.
combined_trips	View that aligns columns and appends both services.
duck_clean_df	SQL view with shared location and timestamp filters.
df_cleaned	Pandas copy of the cleaned result; used by plots and models.
df	Initially an enrichment copy; later reused and overwritten.
taxi_weather_holiday_df	Final enriched pandas result; not selected by the model cells.

Read the data correctly

Common field names established in Cell 23

Common field	Yellow source	Rideshare source
pickup_datetime	tpep_pickup_datetime	pickup_datetime
dropoff_datetime	tpep_dropoff_datetime	dropoff_datetime
pickup_location_id	PULocationID	PULocationID
dropoff_location_id	DOLocationID	DOLocationID
trip_miles	trip_distance	trip_miles
total_charged	total_amount	Sum of selected fare components
airport_fee	Airport_fee	airport_fee

Sample size and saved counts

For the saved August 2024 through August 2026 range, the initial sampling target is 25 months x 2 services x 5,000 rows = 250,000 trips. Subsequent filters can reduce that count. One saved cleaned result contains 249,132 rows. Other cells retain 249,550, 100,000, or 500,000-row outputs from different states. They cannot all describe one clean run of the current source.

Missing values have different meanings

A rideshare passenger_count is NULL because the combined query inserted it; a missing weather value is a different issue; an unmapped zone is another. One blanket dropna rule cannot distinguish these meanings. Structural missingness should be addressed through feature selection, service-specific models, or an explicit encoding/imputation strategy appropriate to the research question.

Persistence is not the same as memory

Raw Parquet files survive in the intended Drive folder. Sample files and the DuckDB catalog are runtime-local. Views describe queries; they do not necessarily hold a complete copy of their result. Calling .df() materializes the returned rows in pandas. Additional dataframe copies consume additional memory, and the current export cell does not write a durable cleaned dataset.

Cells 1-10: Setup and downloads

Cell 1 | Introduce the source datasets

MARKDOWN / DOCUMENTATION

CELL 1 / ORIGINAL MARKDOWN

```
# Reading in the yellow taxis and the for-hire-vehicles (FHV) dataset
```

What Cell 1 does

This Markdown title introduces the yellow taxi and for-hire vehicle datasets. The files actually selected later use the fhvhv prefix, meaning high-volume for-hire vehicle records. Each record generally describes one passenger trip. The eventual model target is the recorded trip charge.

Cell 2 | Import the project libraries

CODE / ACTIVE SOURCE

CELL 2 / ORIGINAL CODE

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import duckdb
import time
import numpy as np
import requests
import os
from io import BytesIO
from datetime import datetime
from pathlib import Path

```

Reads: Installed Python packages
Creates or changes: Names such as pd, sns, plt, duckdb, np, requests, datetime, and Path

What Cell 2 does

pandas supplies dataframes, joins, grouping, datetime accessors, and cleaning. DuckDB supplies the SQL engine that reads Parquet files. requests performs HTTP downloads and weather requests. pathlib.Path builds and inspects file paths; datetime parses the month inputs.

Seaborn and Matplotlib produce and format plots. NumPy supplies numerical operations, including the cumulative sum used in the PCA section. os appears in older file-download code. time and BytesIO are imported but not meaningfully used in the active workflow. Importing a package makes its functions available; it does not load trip data.

Cell 2: key statements and connections

Lines 1-11. The as aliases are shorter names for modules. A from ... import statement brings one named class or function into the current namespace. These names are reused throughout the notebook, so the import cell must run before dependent cells.

Cell 3 | Describe persistent storage and temporary database state

MARKDOWN / DOCUMENTATION

CELL 3 / ORIGINAL MARKDOWN

```
**Pipeline:** Use Google Drive to persist monthly raw (unsampled) Parquet files so they
don't disappear when runtime resets. After entering the date range, check to see if file for
that month already exists. If not, download it from url.

DuckDB runs in memory and creates temporary SQL views for the selected months only. For now,
.duckdb files are not maintained, but we can look into materializing frequently used or
computationally expensive query results into persistent DuckDB tables in the future to avoid
redundant processing and improve performance.
```

What Cell 3 does

The plan is to retain full monthly Parquet files in Google Drive so they survive a Colab runtime reset. DuckDB is opened without a database filename, so its database catalog is temporary. The views can be rebuilt from saved Parquet files.

The note about materializing results refers to saving a computed result as a real table or file. A view stores a query definition; querying it can repeat the underlying work. Materialization can avoid repeating an expensive query, at the cost of storing another result and keeping it up to date.

Cell 4 | Document the raw-data folder

MARKDOWN / DOCUMENTATION

CELL 4 / ORIGINAL MARKDOWN

```
**MyDrive/CIS5450_GroupProject/**

> **taxi_data/** - Original, full Parquet files
  - e.g. yellow_tripdata_2024-08.parquet

```

What Cell 4 does

This Markdown cell documents the intended Google Drive directory, MyDrive/CIS5450_GroupProject/taxi_data, and an example monthly filename. It does not create a folder or download anything. The directory is created by the code in Cell 6.

Cell 5 | Describe the intended data flow

MARKDOWN / DOCUMENTATION

CELL 5 / ORIGINAL MARKDOWN

```
**Data flow:** Raw (unsampled) monthly Parquet files in Google Drive → sampled Parquet files
in Colab runtime → yellow_taxi and fhvhv in-memory views → combined_trips → duck_clean_df →
df_cleaned.
```

What Cell 5 does

The documented sequence is raw monthly files, sampled files, separate service views, a combined view, a cleaned view, and a pandas dataframe. The design reduces the working dataset before pandas loads it into Python memory. Later cells add a separate enrichment branch that this early description does not fully capture.

Cell 6 | Mount Google Drive and set sample parameters

CODE / ACTIVE SOURCE

CELL 6 / ORIGINAL CODE

```
# Google Drive stores raw (unsampled) Parquet files
# Colab runtime stores sample files

from google.colab import drive
drive.mount('/content/drive')

DATA_FOLDER = Path('/content/drive/MyDrive/CIS5450_GroupProject/taxi_data')
SAMPLE_FOLDER = Path('/content/taxi_samples')
DATA_FOLDER.mkdir(parents=True, exist_ok=True)
SAMPLE_FOLDER.mkdir(parents=True, exist_ok=True)

SAMPLE_SIZE = 5000
RANDOM_SEED = 42

print('Raw files (persistent):', DATA_FOLDER)
print('Samples (runtime only):', SAMPLE_FOLDER)
```

Reads: Colab Drive access; Path from Cell 2
Creates or changes: DATA_FOLDER, SAMPLE_FOLDER, SAMPLE_SIZE, RANDOM_SEED; directories

What Cell 6 does

drive.mount connects Google Drive to a Colab session. DATA_FOLDER points to persistent raw files, while SAMPLE_FOLDER points to temporary sample files inside the runtime. mkdir(parents=True, exist_ok=True) creates missing parent directories and tolerates folders that already exist.

SAMPLE_SIZE requests 5,000 rows from each monthly file. RANDOM_SEED supplies a fixed seed to the sampling procedure. These are per-file settings, not a 5,000-row limit for the entire project.

Cell 6: key statements and connections

Lines 7-10. The / operator between Path objects builds a path; it is not arithmetic division. Here the initial paths are constructed directly from strings, then mkdir creates their directories.

Lines 12-13. These settings are read by both sampling loops. Changing SAMPLE_SIZE affects each selected source file rather than setting a global row limit.

> Saved error: ModuleNotFoundError: No module named 'google'. This setup assumes Google Colab. On a local Jupyter kernel, replace the Drive mount and /content paths with local directories. A failure at the import also prevents the later assignments in this cell from running.

Cell 7 | Construct monthly download URLs

CODE / ACTIVE SOURCE

CELL 7 / ORIGINAL CODE

```
# Ask the user for the start and end months
start_input = input("Enter start month (YYYY-MM): ")
end_input = input("Enter end month (YYYY-MM): ")

# Convert inputs to dates
start_date = datetime.strptime(start_input, "%Y-%m")
end_date = datetime.strptime(end_input, "%Y-%m")

# Validate the range
if start_date > end_date:
    raise ValueError("Start month must be before or equal to end month.")

# Generate every month in the range, inclusive
yellow_urls = []
fhv_urls = []

year = start_date.year
month = start_date.month

while (year, month) <= (end_date.year, end_date.month):
    #yellow taxi data
    yellow_urls.append(
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/"
        f"yellow_tripdata_{year}-{month:02d}.parquet"
    )

    #for-hire / ride-share data
    fhv_urls.append(
        f"https://d37ci6vzurychx.cloudfront.net/trip-data/"
        f"fhvhv_tripdata_{year}-{month:02d}.parquet"
    )

    # Move to the next month
    if month == 12:
        year += 1
        month = 1
    else:
        month += 1

print(f"\nCreated {len(yellow_urls)} yellow cab monthly URLs and {len(fhv_urls)} for-hire vehicles months URLs.")

# for url in yellow_urls:
#     print(url)

# for url in fhv_urls:
#     print(url)

cab_urls = []
cab_urls = yellow_urls + fhv_urls
for url in cab_urls:
  print(url)

```

Reads: User-supplied month strings; datetime
Creates or changes: yellow_urls, fhv_urls, cab_urls, start_input, end_input

What Cell 7 does

Two input calls request start and end months in YYYY-MM format. datetime.strptime parses them, and a comparison rejects a start date later than the end date. A while loop visits every month inclusively and handles the transition from December to January.

The code creates a yellow taxi URL and a rideshare URL for each month. Formatting month as :02d produces values such as 08 rather than 8. cab_urls concatenates the two URL lists; it does not combine trip data.

The saved inputs cover August 2024 through August 2026: 25 months, 25 yellow files, and 25 rideshare files. Constructing a URL does not verify that the corresponding remote file exists. Invalid text input also raises a parsing error rather than prompting again.

Cell 7: key statements and connections

Line 20. The pair (year, month) is compared lexicographically, first by year and then by month. That makes the inclusive end-month condition work across year boundaries.

Lines 22-30. Adjacent f-strings inside each append call concatenate into one URL. Expressions in braces substitute the current year and month.

Lines 48-51. Assigning an empty cab_urls immediately before replacing it is redundant. The final for loop only prints URLs; Cell 8 performs the actual downloads.

Cell 8 | Download missing full monthly files

CODE / ACTIVE SOURCE

CELL 8 / ORIGINAL CODE

```
# Download any missing full monthly files to Google Drive.
import requests
from pathlib import Path

# Download any missing full Parquet files to Google Drive
for url in cab_urls:
    filename = Path(url).name
    raw_file = DATA_FOLDER / filename

    # Skip files that already exist in Google Drive
    if raw_file.exists() and raw_file.stat().st_size > 0:
        print(f"Already downloaded: {filename}")
        continue

    # Download missing files
    print(f"Downloading: {filename}")

    response = requests.get(url, timeout=120)
    response.raise_for_status()

    with open(raw_file, "wb") as f:
        f.write(response.content)

    print(f"Saved: {filename}")
```

Reads: cab_urls and DATA_FOLDER
Creates or changes: Full monthly files saved under DATA_FOLDER

What Cell 8 does

For each URL, Path(url).name extracts the filename and DATA_FOLDER / filename creates the destination. The cell skips a file that already exists and has a positive byte size. Otherwise requests.get downloads it, raise_for_status rejects HTTP failures, and a binary write saves response.content.

This avoids redundant downloads, but a nonempty file is not proof of a complete, valid Parquet file. The download is also buffered in memory before writing because the code accesses response.content. A failed request stops this loop; retry and resumable-download logic are not implemented.

Cell 8: key statements and connections

Lines 11-13. continue immediately starts the next loop iteration. A file that passes the existence/size check never reaches requests.get.

Lines 21-22. The with block closes the file after writing, including if an exception occurs while inside the block. wb means write binary bytes, which is appropriate for Parquet.

Cell 9 | Explain why the old download path was disabled

MARKDOWN / DOCUMENTATION

CELL 9 / ORIGINAL MARKDOWN

```
Commented out the below cell for now since the files persisted here seem to be tied to the
runtime only and disappear if you change your runtime.
```

What Cell 9 does

This Markdown note explains that runtime-local files can disappear when the Colab runtime changes. That is the motivation for keeping the raw files in Drive. It has no computational effect.

Cell 10 | Preserve the older local download implementation

COMMENTED CODE / INACTIVE

CELL 10 / ORIGINAL CODE

```
#make the temporary folder for the data and put the parquet files there
# os.makedirs("taxi_data", exist_ok=True)

# for url in cab_urls:
#     filename = os.path.join("taxi_data", url.split("/")[-1])

#     if not os.path.exists(filename):
#         print("Downloading", filename)

#         r = requests.get(url)
#         r.raise_for_status()

#         with open(filename, "wb") as f:
#             f.write(r.content)
```

What Cell 10 does

All source lines are commented out. If enabled, this code would create a relative taxi_data folder and download files there when absent. It is retained as historical work rather than the active download route.

Saved download messages remain beneath the cell. Notebook outputs are stored separately from source, so commenting out source does not erase its earlier output.

Cells 11-22: Sampling and SQL views

Cell 11 | Filter and sample each yellow taxi month

CODE / ACTIVE SOURCE

CELL 11 / ORIGINAL CODE

```
# Generate Yellow Taxi samples for selected months in Colab runtime
con = duckdb.connect()  # In-memory only - no .duckdb file

for url in yellow_urls:
    raw_file = DATA_FOLDER / Path(url).name
    sample_file = SAMPLE_FOLDER / f"{raw_file.stem}_sample.parquet"
    if not raw_file.is_file():
        raise FileNotFoundError(f"Missing raw file: {raw_file}")

    # Replace any previous sample for this selected month.
    sample_file.unlink(missing_ok=True)

    input_path = str(raw_file).replace("'", "''")
    output_path = str(sample_file).replace("'", "''")
    con.execute(f"""
        COPY (
            WITH filtered_data AS (
                SELECT * FROM read_parquet('{input_path}')
                WHERE trip_distance > 0
                  AND total_amount > 0
                  AND passenger_count > 0
                  AND payment_type <> 4
                  AND PULocationID <> 265
                  AND DOLocationID <> 265
            )
            SELECT * FROM filtered_data USING SAMPLE {SAMPLE_SIZE} ROWS (reservoir,
{RANDOM_SEED})
        ) TO '{output_path}' (FORMAT PARQUET)
    """)
    print(f"Saved sample: {sample_file.name}")
```

Reads: yellow_urls, raw files, folders, sampling settings
Creates or changes: con; yellow monthly sample Parquet files

What Cell 11 does

duckdb.connect() opens the temporary database connection. For each selected yellow file, the cell checks that the raw file exists, removes an older sample, and escapes apostrophes in path strings before inserting them into SQL.

The common table expression, filtered_data, retains positive distance, positive total charge, positive passenger count, payment type other than 4, and endpoint IDs other than 265. The sample is drawn from this filtered relation, so these eligibility rules apply before sampling.

Reservoir sampling selects a fixed number of eligible rows, assuming that many exist, without loading the whole source into a pandas dataframe. COPY writes the result to a sample Parquet file. With 25 sufficiently

large monthly files, the initial yellow sample is 125,000 trips. Exact seed reproducibility also depends on execution conditions, including threading [2].

Cell 11: key statements and connections

Lines 5-6. Path(url).name keeps the filename; raw_file.stem removes its extension. The sample name therefore retains the source month and adds _sample.parquet.

Lines 15-28. Python builds a SQL string and hands it to DuckDB. Python controls the per-month loop; DuckDB performs record filtering, sampling, and export inside each iteration.

Lines 17-27. WITH names the eligible relation. SELECT * retains all source columns in the sample. COPY writes those selected rows, rather than creating a persistent DuckDB trip table.

Cell 12 | Filter and sample each rideshare month

CODE / ACTIVE SOURCE

CELL 12 / ORIGINAL CODE

```
# Regenerate FHVHV samples for selected months in Colab runtime.
for url in fhv_urls:
    raw_file = DATA_FOLDER / Path(url).name
    sample_file = SAMPLE_FOLDER / f"{raw_file.stem}_sample.parquet"
    if not raw_file.is_file():
        raise FileNotFoundError(f"Missing raw file: {raw_file}")

    # Replace any previous sample for this selected month.
    sample_file.unlink(missing_ok=True)

    input_path = str(raw_file).replace("'", "''")
    output_path = str(sample_file).replace("'", "''")
    con.execute(f"""
        COPY (
            WITH filtered_data AS (
                SELECT * FROM read_parquet('{input_path}')
                WHERE trip_miles > 0
                  AND PULocationID <> 265
                  AND DOLocationID <> 265
            )
            SELECT * FROM filtered_data USING SAMPLE {SAMPLE_SIZE} ROWS (reservoir,
{RANDOM_SEED})
        ) TO '{output_path}' (FORMAT PARQUET)
    """)
    print(f"Saved sample: {sample_file.name}")
```

Reads: fhv_urls, con, raw files, folders, sampling settings
Creates or changes: Rideshare monthly sample Parquet files

What Cell 12 does

This repeats the same file-by-file sampling and Parquet export for fhvhv records. Its filters require trip_miles > 0 and both endpoint IDs to differ from 265. Rideshare uses trip_miles where yellow data uses trip_distance.

The rideshare eligibility rules are less restrictive than the yellow rules. For example, this cell does not validate the eventual reconstructed total charge. The initial target is another 125,000 trips for the saved 25-month range.

Equal sample counts per month and service do not preserve actual NYC trip shares. A month with twice as many underlying trips still contributes 5,000 sampled trips. Treat pooled counts and averages as sample statistics unless population weights are deliberately introduced.

Cell 12: key statements and connections

Lines 13-23. This loop reuses con created in Cell 11. Re-running it replaces selected sample files, because unlink executes before COPY.

Cell 13 | Keep the old yellow sampling approach as a reference

COMMENTED CODE / INACTIVE

CELL 13 / ORIGINAL CODE

```
#Runtime-only version

# con = duckdb.connect()

# # data_folder = DATA_FOLDER
# # sample_folder = SAMPLE_FOLDER

# # Source files on your current working directory / Drive
# data_folder = Path("taxi_data")
# sample_folder = Path("taxi_samples")
# sample_folder.mkdir(parents=True, exist_ok=True)

# files = sorted(data_folder.glob("yellow*.parquet"))

# print(f"Found {len(files)} files")

# for file in files:
#     output_file = sample_folder / f"{file.stem}_sample.parquet"

#     # Skip files already sampled, useful if Colab crashes
#     if output_file.exists():
#         print(f"Skipping existing sample: {file.name}")
#         continue

#     # Escape single quotes in paths for SQL
#     input_path = str(file.resolve()).replace("'", "''")
#     output_path = str(output_file.resolve()).replace("'", "''")

#     query = f"""
#         COPY (
#             WITH filtered_data AS (
#                 SELECT *
#                 FROM read_parquet('{input_path}')
#                 WHERE trip_distance > 0
#                   AND total_amount > 0
#                   AND passenger_count > 0
#                   AND payment_type <> 4
#                   AND PULocationID <> 265
#                   AND DOLocationID <> 265
#             )
#             SELECT *
#             FROM filtered_data
#             USING SAMPLE 5000 ROWS
#         )
#         TO '{output_path}'
#         (FORMAT PARQUET);
#     """

#     try:
```

CELL 13 / ORIGINAL CODE / CONTINUED

```
#         con.execute(query)
#         print(f"Saved sample: {output_file.name}")
#     except Exception as e:
#         print(f"Failed: {file.name}: {e}")
#         raise

# print("Sampling complete!")

# print("Combing all the yellow files")
# query_yellow = """
#     SELECT *
#     FROM read_parquet('taxi_samples/yellow*_sample.parquet')
# """

# yellow_sampled_data = con.execute(query_yellow)

# # print(df_yellow.shape)
```

What Cell 13 does

The entire cell is commented out. Its older approach searches a local directory using a wildcard, samples matching yellow files, skips samples that already exist, and queries the resulting files.

Unlike the active method, file discovery is driven by whatever is present in the directory rather than the current month-selection lists. The shown old sampling SQL also lacks the explicit seed used by the active version. It has no effect while commented out.

Cell 14 | Keep the old rideshare sampling approach as a reference

COMMENTED CODE / INACTIVE

CELL 14 / ORIGINAL CODE

```
#Runtime-only version

# #Getting fhvhv files

# # con = duckdb.connect()

# # Source files on your current working directory / Drive
# data_folder = Path("taxi_data")
# sample_folder = Path("taxi_samples")
# sample_folder.mkdir(parents=True, exist_ok=True)

# files = sorted(data_folder.glob("fhvhv*.parquet"))

# print(f"Found {len(files)} files")

# for file in files:
#     output_file = sample_folder / f"{file.stem}_sample.parquet"

#     # Skip files already sampled, useful if Colab crashes
#     if output_file.exists():
#         print(f"Skipping existing sample: {file.name}")
#         continue

#     # Escape single quotes in paths for SQL
#     input_path = str(file.resolve()).replace("'", "''")
#     output_path = str(output_file.resolve()).replace("'", "''")

#     query = f"""
#         COPY (
```

CELL 14 / ORIGINAL CODE / CONTINUED

```
#             WITH filtered_data AS (
#                 SELECT *
#                 FROM read_parquet('{input_path}')
#                 WHERE trip_miles > 0
#                   AND PULocationID <> 265
#                   AND DOLocationID <> 265
#             )
#             SELECT *
#             FROM filtered_data
#             USING SAMPLE 5000 ROWS
#         )
#         TO '{output_path}'
#         (FORMAT PARQUET);
#     """

#     try:
#         con.execute(query)
#         print(f"Saved sample: {output_file.name}")
#     except Exception as e:
#         print(f"Failed: {file.name}: {e}")
#         raise

# print("Sampling complete!")

# print("Combing all the fhvhv files")
# query_fhvhv = """
#     SELECT *
#     FROM read_parquet('taxi_samples/fhvhv*_sample.parquet')
# """

# fhvhv_sampling = con.execute(query_fhvhv)

# # print(df_fhvhv.shape)
```

What Cell 14 does

This is the rideshare counterpart of Cell 13. It would find matching fhvhv files locally, create sample files, and query them. All code is commented out, so visible progress messages are historical outputs.

Cell 15 | Build the exact sample-file lists

CODE / ACTIVE SOURCE

CELL 15 / ORIGINAL CODE

```
 # Get the sample files for the selected months
yellow_samples = []
fhvhv_samples = []

for url in yellow_urls:
    filename = Path(url).stem + "_sample.parquet"
    yellow_samples.append(str(SAMPLE_FOLDER / filename))

for url in fhv_urls:
    filename = Path(url).stem + "_sample.parquet"
    fhvhv_samples.append(str(SAMPLE_FOLDER / filename))

# Check that all required samples exist in Google Drive
for file in yellow_samples + fhvhv_samples:
    if not Path(file).is_file() or Path(file).stat().st_size == 0:
```

CELL 15 / ORIGINAL CODE / CONTINUED

```
        raise FileNotFoundError(f"Missing or empty sample: {file}")

print("All requested samples are available in the Colab runtime!")
```

Reads: Month URL lists and SAMPLE_FOLDER
Creates or changes: yellow_samples and fhvhv_samples path lists

What Cell 15 does

For each selected monthly URL, the code constructs the matching _sample.parquet filename and joins it to SAMPLE_FOLDER. It produces separate lists for yellow and rideshare samples.

The validation loop rejects missing or empty sample files. This explicit selection prevents an unrelated sample left over from another experiment from silently entering the analysis. A comment mentions Google Drive, but the active paths point to the runtime sample directory.

Cell 15: key statements and connections

Lines 5-11. The resulting lists contain string paths, not loaded dataframes. DuckDB opens these paths when constructing and querying its relations.

Lines 14-16. The or condition rejects either a missing file or a zero-byte file. Short-circuit evaluation avoids calling stat on a missing path.

Cell 16 | Create yellow_taxi and fhvhv SQL views

CODE / ACTIVE SOURCE

CELL 16 / ORIGINAL CODE

```
# Create in-memory DuckDB views

# Yellow Taxi Data
con.from_parquet(yellow_samples).filter("""
    passenger_count > 0
    AND payment_type NOT IN (0, 3, 4, 5, 6)
    AND trip_distance > 0
    AND RatecodeID IS NOT NULL
""").create_view("yellow_taxi", replace=True)

# FHVHV Data
con.from_parquet(fhvhv_samples).create_view("fhvhv", replace=True)

print(f"In-memory DuckDB views created for the selected months: {start_input} to
{end_input}")
```

Reads: con and the two sample path lists
Creates or changes: yellow_taxi and fhvhv views in the connection

What Cell 16 does

con.from_parquet reads the selected file lists as DuckDB relations. The yellow relation receives additional filters: positive passenger count and distance, payment type outside 0/3/4/5/6, and nonmissing RatecodeID. create_view gives the relation a reusable SQL name and replaces an older view of that name.

The rideshare relation is exposed as fhvhv without additional filtering here. Yellow rows can now fall below 5,000 per month because some exclusions occur after the initial sample was selected.

Monthly file schemas can differ across years. The code does not explicitly request union_by_name=True, which DuckDB provides to reconcile different or missing columns by name [3]. Schema compatibility should be verified for the selected files.

Cell 16: key statements and connections

Lines 4-9. This is method chaining: construct a relation, attach a SQL predicate, then register a named view. replace=True allows the same view name to be rebuilt in an existing session.

Line 12. The rideshare view has no corresponding additional filter in this cell. Cleaning is therefore not identical between services at this point.

Cell 17 | Show an inactive SQL alternative for creating views

COMMENTED CODE / INACTIVE

CELL 17 / ORIGINAL CODE

```
# Use DuckDB to parameterize the Parquet file list and store views only in memory

# con.execute("""
#     CREATE OR REPLACE VIEW yellow_taxi AS
#     SELECT * FROM read_parquet($files)
#     WHERE passenger_count > 0
#       AND payment_type NOT IN (0, 3, 4, 5, 6)
#       AND trip_distance > 0
#       AND RatecodeID IS NOT NULL
# """, {"files": yellow_samples})

# con.execute("""
#     CREATE OR REPLACE VIEW fhvhv AS
#     SELECT *
#     FROM read_parquet($files)
# """, {"files": fhvhv_samples})

# print('In-memory DuckDB views ready for selected months.')
```

What Cell 17 does

This commented cell is an alternative attempt to create the same two views using SQL and a supplied list of file paths. Cell 16 is the active implementation. Do not assume the commented parameterized CREATE VIEW example has been validated merely because it remains in the notebook.

Cell 18 | Inspect each source schema

CODE / ACTIVE SOURCE

CELL 18 / ORIGINAL CODE

```
print("Yellow Taxi:")
print(con.execute("DESCRIBE yellow_taxi").df())

print("FHVHV:")
print(con.execute("DESCRIBE fhvhv").df())
```

What Cell 18 does

DESCRIBE returns column names, SQL types, and schema information for the two views. Converting those results to pandas makes them easy to inspect.

This is the place to identify equivalent fields with different names, fields found in only one service, and datatype mismatches. A nullable column definition indicates that NULL is permitted; it does not provide a count of actual missing values.

Cell 19 | List the named database relations

CODE / ACTIVE SOURCE

CELL 19 / ORIGINAL CODE

```
1  con.execute("SHOW TABLES").fetchdf()
```

What Cell 19 does

SHOW TABLES displays the relations available through this connection. The saved result contains yellow_taxi and fhvhv. Despite the command name, these particular objects are views rather than separately materialized trip tables.

Cell 20 | Preview yellow taxi records

CODE / ACTIVE SOURCE

CELL 20 / ORIGINAL CODE

```
1  # Confirm that there is data in the yellow taxis
2  con.execute("""SELECT * from yellow_taxi LIMIT 5""").fetch_df()
```

What Cell 20 does

SELECT * FROM yellow_taxi LIMIT 5 retrieves five rows and fetch_df converts them to pandas for display. This verifies basic access and gives a visual check of field values. LIMIT does not draw a new random sample, and without ORDER BY there is no promised meaningful order.

Cell 21 | Preview rideshare records

CODE / ACTIVE SOURCE

CELL 21 / ORIGINAL CODE

```
1  # Confirm that there is data in the for-hire vehciles data set
2
3  con.execute("""SELECT * from fhvhv LIMIT 5""").fetch_df()
```

What Cell 21 does

The equivalent five-row query for fhvhv shows the different schema: license identifiers, request and arrival times, pickup and drop-off times, and other rideshare-specific fields. A plausible preview is useful, but it does not establish data quality across every row.

Cell 22 | Retain an optional full-file inspection

COMMENTED CODE / INACTIVE

CELL 22 / ORIGINAL CODE

```
# If you want to read the original parquet file to check data quality you can do it below
# df = pd.read_parquet("taxi_data/fhvhv_tripdata_2024-10.parquet")
```

What Cell 22 does

The commented pandas read_parquet call would load a complete monthly rideshare file directly. This is useful for investigating a source file, but bypasses the sample and can use much more memory. The current cell does not execute any read.

Cells 23-32: Combining and cleaning trips

Cell 23 | Align the two schemas and stack trips

CODE / ACTIVE SOURCE

CELL 23 / ORIGINAL CODE

```
all_data = con.execute("""
CREATE OR REPLACE VIEW combined_trips AS

SELECT
    'yellow' AS service_type,

    VendorID,
    passenger_count,
    RatecodeID,
    store_and_fwd_flag,

    tpep_pickup_datetime AS pickup_datetime,
    tpep_dropoff_datetime AS dropoff_datetime,

    PULocationID AS pickup_location_id,
    DOLocationID AS dropoff_location_id,

    payment_type,
    fare_amount,
    extra,
    mta_tax,
    tip_amount,
    tolls_amount,
    improvement_surcharge,
    total_amount as total_charged,

    trip_distance AS trip_miles,
    congestion_surcharge,
    Airport_fee AS airport_fee,

    -- FHVHV-only columns
    NULL::VARCHAR AS hvfhs_license_num,
    NULL::VARCHAR AS dispatching_base_num,
    NULL::VARCHAR AS originating_base_num,
    NULL::TIMESTAMP AS request_datetime,
    NULL::TIMESTAMP AS on_scene_datetime,
```

CELL 23 / ORIGINAL CODE / CONTINUED

```
    NULL::BIGINT AS trip_time,
    NULL::DOUBLE AS base_fare,
    NULL::DOUBLE AS tolls,
    NULL::DOUBLE AS bcf,
    NULL::DOUBLE AS sales_tax,
    NULL::DOUBLE AS tips,
    NULL::DOUBLE AS driver_pay,
    NULL::VARCHAR AS shared_request_flag,
    NULL::VARCHAR AS shared_match_flag,
    NULL::VARCHAR AS access_a_ride_flag,
    NULL::VARCHAR AS wav_request_flag,
    NULL::VARCHAR AS wav_match_flag

FROM yellow_taxi

UNION ALL

SELECT
    'fhvhv' AS service_type,

    -- Yellow-only columns
    NULL::INTEGER AS VendorID,
    NULL::BIGINT AS passenger_count,
    NULL::BIGINT AS RatecodeID,
    NULL::VARCHAR AS store_and_fwd_flag,

    pickup_datetime,
    dropoff_datetime,

    PULocationID AS pickup_location_id,
    DOLocationID AS dropoff_location_id,

    NULL::BIGINT AS payment_type,
    NULL::DOUBLE AS fare_amount,
    NULL::DOUBLE AS extra,
    NULL::DOUBLE AS mta_tax,
    NULL::DOUBLE AS tip_amount,
    NULL::DOUBLE AS tolls_amount,
    NULL::DOUBLE AS improvement_surcharge,

    (COALESCE(base_passenger_fare, 0) + COALESCE(tolls, 0) + COALESCE(bcf, 0)
      + COALESCE(sales_tax) + COALESCE(congestion_surcharge, 0) + COALESCE(airport_fee, 0))
AS total_charged,
    trip_miles,
    congestion_surcharge,
    airport_fee,

    hvfhs_license_num,
    dispatching_base_num,
    originating_base_num,
    request_datetime,
    on_scene_datetime,
    trip_time,
    base_passenger_fare AS base_fare,
    tolls,
    bcf,
    sales_tax,
    tips,
    driver_pay,
    shared_request_flag,
    shared_match_flag,
    access_a_ride_flag,
```

CELL 23 / ORIGINAL CODE / CONTINUED

```
    wav_request_flag,
    wav_match_flag

FROM fhvhv;
""")
```

Reads: yellow_taxi and fhvhv views
Creates or changes: combined_trips view; all_data holds an execution return value

What Cell 23 does

This creates combined_trips, the central integration view. The yellow SELECT renames tpep_pickup_datetime to pickup_datetime, tpep_dropoff_datetime to dropoff_datetime, endpoint IDs to pickup_location_id/dropoff_location_id, trip_distance to trip_miles, total_amount to total_charged, and Airport_fee to airport_fee.

Each service receives a literal service_type label. Fields that only exist in the other service are supplied as typed NULLs. NULL::BIGINT AS passenger_count gives a rideshare row a missing integer passenger-count field; it does not mean zero passengers. Types are specified so the two SELECT lists remain compatible.

UNION ALL appends the rows vertically. It does not match equivalent yellow and rideshare journeys. Column positions in the two SELECT lists must align, and duplicate records are preserved.

For rideshare, total_charged is reconstructed from base passenger fare, tolls, Black Car Fund charge, sales tax, congestion surcharge, and airport fee. COALESCE(value, 0) substitutes zero for a missing component. However, the sales-tax expression has no fallback: COALESCE(sales_tax) can remain NULL and make the sum NULL.

The target definitions are not fully comparable. Yellow total_amount includes recorded credit-card tips but excludes cash tips; the rideshare sum excludes its tips field. The separate cbd_congestion_fee introduced for 2025 onward is also absent from the reconstructed sum. The target needs an explicit, consistent definition across services and years [4, 5, 6].

The Python variable all_data receives the result of executing CREATE VIEW. It is not the pandas table of all trips.

Cell 23: key statements and connections

Lines 4-50. The first SELECT defines the yellow rows and the shared column order. AS changes names in the result; it does not rename the raw Parquet file columns.

Lines 32-48 and 58-75. Typed NULL placeholders maintain the same number and order of columns across the two SELECT statements. VARCHAR is text, TIMESTAMP is datetime, BIGINT is an integer, and DOUBLE is floating point.

Line 52. UNION ALL stacks both result sets without a matching key. This is fundamentally different from the weather join later in the project.

Lines 77-78. The rideshare total is computed here. Because the formula differs from the yellow total field, merely sharing the name total_charged does not guarantee comparable outcomes.

Line 89. base_passenger_fare is also kept separately as base_fare. That later becomes a potential leakage feature because it is a direct component of the target.

> Target definition and structural missingness are two of the most consequential design issues in the notebook. They affect every later comparison and model.

Cell 24 | Verify the combined schema

CODE / ACTIVE SOURCE

CELL 24 / ORIGINAL CODE

```
# print("Yellow Taxi:")
# print(con.execute("DESCRIBE yellow_taxi").df())

print("all_data:")
print(con.execute("DESCRIBE combined_trips").df())
```

What Cell 24 does

DESCRIBE combined_trips lists the unified column names and SQL types. The saved schema has 37 columns. This is a useful checkpoint for verifying that both SELECT statements created the intended column structure and that renamed fields are available downstream.

Cell 25 | Check coverage by service and pickup month

CODE / ACTIVE SOURCE

CELL 25 / ORIGINAL CODE

```
con.execute("""
SELECT
    YEAR(pickup_datetime) AS year, MONTH(pickup_datetime) as month,
    COUNT(*) AS trips, service_type
FROM combined_trips
GROUP BY year, month, service_type
ORDER BY year, month, service_type
""").fetchdf()
```

What Cell 25 does

The query extracts pickup year and month, groups by those values and service_type, counts rows, and orders the results. Saved monthly counts show 5,000 rideshare trips and slightly fewer yellow trips after the extra yellow filters.

Grouping uses the timestamp stored inside the record, not its source filename. An incorrectly dated trip can appear in an unexpected month or year. These are counts of sampled rows, not total trips in the original monthly files.

Cell 26 | Introduce SQL cleaning

MARKDOWN / DOCUMENTATION

CELL 26 / ORIGINAL MARKDOWN

```
# Cleaning the data in DUCK DB
```

What Cell 26 does

This Markdown title marks the shared cleaning stage after the service schemas have been combined.

Cell 27 | Create the cleaned SQL view

CODE / ACTIVE SOURCE

CELL 27 / ORIGINAL CODE

```
clean_data = con.execute("""
    CREATE OR REPLACE VIEW duck_clean_df AS
    SELECT *
    FROM combined_trips
    WHERE pickup_location_id NOT IN (264, 265)
      AND dropoff_location_id NOT IN (264, 265)
      AND pickup_datetime IS NOT NULL
      AND dropoff_datetime IS NOT NULL
""")
```

Reads: combined_trips view
Creates or changes: duck_clean_df view; clean_data holds an execution return value

What Cell 27 does

duck_clean_df retains records whose pickup and drop-off IDs are not 264 or 265 and whose pickup and drop-off timestamps are not NULL. Missing endpoint IDs are also excluded because a NULL comparison does not evaluate to true in a WHERE filter.

The cell does not check that drop-off follows pickup, that dates fall inside the requested interval, that duration is plausible, or that implied speed is reasonable. It also does not validate every service-specific charge field. The variable clean_data is an execution result; duck_clean_df remains a view despite its dataframe-like name.

Cell 27: key statements and connections

Lines 3-8. SELECT * retains the full unified schema while WHERE removes rows. Cleaning changes the eligible observations, not the set of selected columns.

Cell 28 | Count cleaned records

CODE / ACTIVE SOURCE

CELL 28 / ORIGINAL CODE

```
# Count how many rows of data we have after cleaning
con.execute(f"""
SELECT COUNT(*)
FROM duck_clean_df
""").fetchone()
```

What Cell 28 does

COUNT(*) returns the number of rows that satisfy the view definition. fetchone returns one tuple; the saved value (249132,) is a one-element tuple containing 249,132. It is a saved execution result, not a newly reproduced count.

Cell 29 | Materialize cleaned records as pandas

CODE / ACTIVE SOURCE

CELL 29 / ORIGINAL CODE

```
# Convert to pandas db
df_cleaned = con.execute("""
  SELECT *
  FROM duck_clean_df
""").df()

df_cleaned.info()
```

Reads: duck_clean_df view
Creates or changes: df_cleaned pandas dataframe

What Cell 29 does

The SELECT queries duck_clean_df and .df() brings the returned records into a pandas dataframe named df_cleaned. This is where the working trip table is materialized in Python memory.

info() reports row count, column names, nonmissing counts, types, and estimated memory use. One saved result has 249,132 rows: 124,132 yellow records and 125,000 rideshare records. Many columns are only populated for one service.

Do not interpret every NULL as a bad trip. Much of this missingness was deliberately created in Cell 23. Dropping all rows that contain any NULL would be inappropriate for this unified structure.

Cell 29: key statements and connections

Lines 2-5. The .df() call is the transition from the SQL layer into a pandas table. Later pandas operations do not automatically edit the DuckDB views or the source files.

Cell 30 | Create the enrichment copy

CODE / ACTIVE SOURCE

CELL 30 / ORIGINAL CODE

```
# To be used to merge w/Weather and Events datasets
df = df_cleaned.copy()
```

Reads: df_cleaned
Creates or changes: Separate pandas copy named df

What Cell 30 does

df = df_cleaned.copy() creates a separate dataframe for weather and other additions. This is a branch in the pipeline. New columns added to df do not automatically appear in df_cleaned. Later model cells continue using df_cleaned, which is why the enrichment does not reach the model.

Cell 31 | Preserve a planned Parquet export

COMMENTED CODE / INACTIVE

CELL 31 / ORIGINAL CODE

```
# Putting the cleaned data in parquet file
# con.execute("""
#     COPY (
#         SELECT
#            *
#         FROM duck_clean_df
#     TO 'taxi_ml_data.parquet'
#     (FORMAT PARQUET)
# """)
```

What Cell 31 does

The commented SQL is intended to export cleaned trips to taxi_ml_data.parquet. It currently writes nothing. If uncommented as shown, it also needs a closing parenthesis for COPY (SELECT ...) before TO.

This matters because the later Spark cell tries to read that exact filename. A proposed export and an existing exported file are different states.

Cell 32 | Empty cell

EMPTY / NO EFFECT

The original cell is empty.

What Cell 32 does

This cell contains no executable statements or explanatory content. It has no effect.

Cells 33-39: Weather enrichment

Cell 33 | Introduce weather enrichment

MARKDOWN / DOCUMENTATION

CELL 33 / ORIGINAL MARKDOWN

```
## Weather Data
```

What Cell 33 does

This Markdown title begins the weather-data retrieval and joining stage.

Cell 34 | Request historical weather-model data

CODE / ACTIVE SOURCE

CELL 34 / ORIGINAL CODE

```
import requests

# https://open-meteo.com/en/docs/historical-weather-api
# https://open-meteo.com/

#url = "https://archive-api.open-meteo.com/v1/era5"

url = "https://historical-forecast-api.open-meteo.com/v1/forecast"

params = {
    "latitude": 40.7128, # NYC
    "longitude": -74.0060,
    "start_date": "2024-08-01",
    "end_date": "2026-08-31",
    "minutely_15": [
        "temperature_2m", # air temperature at 2 meters above ground level (Fahrenheit)
        "precipitation", # Total precipitation in mm (rain or snow)
        "rain", # Rainfall in mm
        "snowfall", # Snowfall in cm
        "wind_speed_10m", # Wind speed at 10 m
        "weather_code" # Weather condition code (general weather classification)
    ],
    "temperature_unit": "fahrenheit",
    "timezone": "America/New_York"
}

response = requests.get(url, params=params, timeout=60)

# Check whether an HTTP request was successful
response.raise_for_status()
data = response.json()
```

Reads: Weather endpoint and hardcoded request parameters
Creates or changes: response and parsed JSON data

What Cell 34 does

The active URL is Open-Meteo's Historical Forecast API. The request uses one NYC coordinate, latitude 40.7128 and longitude -74.0060, for August 1, 2024 through August 31, 2026. These dates are hardcoded rather than derived from the taxi month inputs.

The requested 15-minute fields are air temperature at 2 meters, precipitation, rain, snowfall, wind speed at 10 meters, and a weather-condition code. The request specifies Fahrenheit and America/New_York. Wind speed is not overridden, so the documented default is kilometers per hour [7, 8].

requests.get sends the request, raise_for_status checks HTTP success, and response.json parses the response into Python structures. The historical-forecast product archives weather-model data; it should not be described as direct readings from a single weather station.

Every trip in the same interval will receive the weather from the same selected coordinate. This is an approximation across NYC geography. A different taxi date range also requires updating the weather request.

Cell 34: key statements and connections

Lines 10-25. params is a Python dictionary of query parameters. requests encodes these into the request. The minutely_15 list selects which weather fields the API returns.

Lines 27-31. The HTTP response object and the parsed JSON dictionary are different objects. Cell 35 reads the dictionary, not the raw HTTP response.

Cell 35 | Turn the weather response into a table

CODE / ACTIVE SOURCE

CELL 35 / ORIGINAL CODE

```
1  weather_df = pd.DataFrame(data["minutely_15"])
2
3  weather_df
```

Reads: data["minutely_15"]
Creates or changes: weather_df

What Cell 35 does

pd.DataFrame(data["minutely_15"]) converts the returned arrays into weather_df. Each row is a weather timestamp with corresponding measurements. At this stage, the unit of observation is one weather interval, not one taxi trip.

Cell 36 | Parse weather timestamps

CODE / ACTIVE SOURCE

CELL 36 / ORIGINAL CODE

```
1  # Convert time to datetime to enable merge with taxi dataset
2  weather_df["time"] = pd.to_datetime(weather_df["time"])
3
4  weather_df
```

Reads: weather_df["time"]
Creates or changes: Same column converted to datetime

What Cell 36 does

pd.to_datetime converts weather time strings into pandas datetime values so they can match the trip-side time key. The code does not explicitly normalize both datasets to timezone-aware UTC timestamps.

Matching local-looking timestamps is not by itself proof of correct daylight-saving alignment across a multiyear range. Audit the API timestamp convention and the taxi convention before relying on exact 15-minute matches.

Cell 37 | Remove incomplete weather intervals

CODE / ACTIVE SOURCE

CELL 37 / ORIGINAL CODE

```
# Remove any rows w/null and na values
weather_cols = [
    "temperature_2m",
    "precipitation",
    "rain",
    "snowfall",
    "wind_speed_10m",
    "weather_code"
]

weather_df = weather_df.dropna(subset=weather_cols)
```

Reads: weather_df and required weather columns
Creates or changes: weather_df reassigned to retained rows

What Cell 37 does

dropna(subset=weather_cols) removes a weather row if any of the six listed measurements is missing. This ensures complete selected weather measurements on the weather side.

Because the subsequent merge is an inner join, trips in removed weather intervals are also lost. Record the number and time distribution of removed intervals so this filtering does not silently bias the retained trips.

Cell 38 | Map weather codes to readable descriptions

CODE / ACTIVE SOURCE

CELL 38 / ORIGINAL CODE

```
# Map Weather Codes
# https://open-meteo.com/en/docs/historical-weather-api

weather_code_map = {
    "0": "Clear sky",
    "1": "Mainly clear",
    "2": "Partly cloudy",
    "3": "Overcast",
    "45": "Fog",
    "48": "Depositing rime fog",
    "51": "Light drizzle",
    "53": "Moderate drizzle",
    "55": "Dense drizzle",
    "56": "Light freezing drizzle",
```

CELL 38 / ORIGINAL CODE / CONTINUED

```
    "57": "Dense freezing drizzle",
    "61": "Slight rain",
    "63": "Moderate rain",
    "65": "Heavy rain",
    "66": "Light freezing rain",
    "67": "Heavy freezing rain",
    "71": "Slight snowfall",
    "73": "Moderate snowfall",
    "75": "Heavy snowfall",
    "77": "Snow grains",
    "80": "Slight rain showers",
    "81": "Moderate rain showers",
    "82": "Violent rain showers",
    "85": "Slight snow showers",
    "86": "Heavy snow showers",
    "95": "Thunderstorm",
    "96": "Thunderstorm with slight hail",
    "97": "Heavy thunderstorm",
    "99": "Thunderstorm with heavy hail"
}

# Convert dictionary keys from strings to integers
weather_code_map = {
    int(code): description
    for code, description in weather_code_map.items()
}

# Map numeric weather codes to their descriptions
weather_df["weather_description"] = (weather_df["weather_code"].map(weather_code_map))

# Display results
weather_df.head()
```

Reads: weather codes and an explicit dictionary
Creates or changes: weather_df["weather_description"]

What Cell 38 does

The dictionary maps codes to labels such as Clear sky, Overcast, and Slight rain. A dictionary comprehension converts string keys to integers, matching the numeric code column. map adds a weather_description column.

A code absent from the dictionary produces a missing description. This is a deterministic lookup, not a learned feature transformation. For modeling, code and description represent the same underlying category; keeping both may be redundant.

Cell 38: key statements and connections

Dictionary conversion and map. The dictionary comprehension changes keys from text such as "3" to integer 3. The map operation then looks up one description per weather row. This prevents a string-versus-integer key mismatch.

Cell 39 | Join weather to each trip by a 15-minute key

CODE / ACTIVE SOURCE

CELL 39 / ORIGINAL CODE

```
1  # Join weather dataset w/taxi dataset on pickup time
2
3  # Convert pickup time in taxi dataset to datetime format and round times down to 15 min
4  df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"])
5  df["15min_pickup_datetime"] = df["pickup_datetime"].dt.floor("15min")
6
7  taxi_weather_df = df.merge(weather_df, left_on="15min_pickup_datetime", right_on="time",
   how="inner").drop(columns=["time"]) # Keep only rows w/complete weather data
8
9  taxi_weather_df.head()
```

Reads: df pickup timestamps and weather_df
Creates or changes: 15min_pickup_datetime in df; new taxi_weather_df

What Cell 39 does

The trip pickup column is parsed as datetime and dt.floor("15min") rounds each pickup down to the start of its interval. A pickup at 14:37:22 therefore gets the key 14:30:00.

merge matches that trip key to weather_df.time. how="inner" keeps only trips with a weather match. Many trips may correctly share one weather interval. The redundant weather-side time column is dropped, and the result is named taxi_weather_df.

Verify that weather timestamps are unique and measure the before/after row count. Duplicate weather keys can multiply trip rows. A missing weather match can remove a trip. Those are different failure modes.

Cell 39: key statements and connections

Lines 4-5. Datetime parsing and interval flooring create compatible join keys. Flooring is not rounding to the nearest interval and does not calculate trip duration.

Line 7. The method chain first merges the two tables and then drops the redundant weather-side time column. The result is assigned to a new dataframe; it does not replace df_cleaned.

Cells 40-51: Geography and holidays

Cell 40 | Introduce the taxi-zone lookup

MARKDOWN / DOCUMENTATION

CELL 40 / ORIGINAL MARKDOWN

```
## Taxi Zone Lookup Table
```

What Cell 40 does

This Markdown title introduces the table that translates location identifiers into readable geographic labels.

Cell 41 | Load location metadata

CODE / ACTIVE SOURCE

CELL 41 / ORIGINAL CODE

```
url = "https://d37ci6vzurychx.cloudfront.net/misc/taxi_zone_lookup.csv"

zones_df = pd.read_csv(url)

zones_df.head()
```

Reads: Taxi-zone lookup CSV URL
Creates or changes: zones_df

What Cell 41 does

pd.read_csv downloads the taxi-zone lookup table. It associates LocationID with Borough, Zone, and service_zone. head() displays examples so you can confirm the lookup structure.

This adds interpretation to the numeric endpoint IDs. It does not add GPS coordinates or calculate physical distance between zones.

Cell 42 | Create zone and borough lookup mappings

CODE / ACTIVE SOURCE

CELL 42 / ORIGINAL CODE

```
# Create mapping from LocationID to zone name
zone_map = zones_df.set_index("LocationID")["Zone"]
borough_map = zones_df.set_index("LocationID")["Borough"]
```

Reads: zones_df
Creates or changes: zone_map and borough_map Series

What Cell 42 does

set_index("LocationID")["Zone"] creates a pandas Series indexed by location ID with zone names as values. The second expression creates an equivalent borough mapping. Each Series can then be used by map to translate a column of IDs without changing the trip row count.

Cell 42: key statements and connections

Both assignments. set_index uses LocationID as the lookup key. Selecting a single column then returns a Series, which map can use like a dictionary of ID-to-label pairs.

Cell 43 | Add names for both trip endpoints

CODE / ACTIVE SOURCE

CELL 43 / ORIGINAL CODE

```
# Add pickup information
taxi_weather_df["pickup_zone"] = taxi_weather_df["pickup_location_id"].map(zone_map)
taxi_weather_df["pickup_borough"] = taxi_weather_df["pickup_location_id"].map(borough_map)

# Add dropoff information
taxi_weather_df["dropoff_zone"] = taxi_weather_df["dropoff_location_id"].map(zone_map)
taxi_weather_df["dropoff_borough"] = taxi_weather_df["dropoff_location_id"].map(borough_map)
```

Reads: taxi_weather_df endpoint IDs and both mappings
Creates or changes: Four zone/borough columns in taxi_weather_df

What Cell 43 does

Four map operations add pickup_zone, pickup_borough, dropoff_zone, and dropoff_borough to taxi_weather_df. IDs absent from a mapping become missing values. These labels are useful for readable plots and grouped comparisons. Numeric location IDs should still be treated as category identifiers rather than ordered quantities.

Cell 44 | Inspect the geographically enriched rows

CODE / ACTIVE SOURCE

CELL 44 / ORIGINAL CODE

```
taxi_weather_df.head()
```

What Cell 44 does

head() displays the first five rows with weather and geographic labels. This checks whether the added columns appear plausible. A preview cannot detect every unmapped ID, so a separate missing-value count remains useful.

Cell 45 | Count trips by pickup borough

CODE / ACTIVE SOURCE

CELL 45 / ORIGINAL CODE

```
taxi_weather_holiday_df["pickup_borough"].value_counts()
```

What Cell 45 does

The intended value_counts operation reports how many sampled trips originate in each borough, omitting missing values by default. However, it reads taxi_weather_holiday_df before that variable is created in Cell 49.

> Fresh-run failure: NameError, unless this variable already exists in kernel memory. Move the check after Cell 49 or use taxi_weather_df here. The saved borough counts show a prior execution state, not proof that the current order works.

Cell 46 | Keep an inactive missing-value diagnostic

COMMENTED CODE / INACTIVE

CELL 46 / ORIGINAL CODE

```
1  #taxi_weather_holiday_df.isna().sum()
```

What Cell 46 does

The commented isna().sum() would count missing values in each enriched column. It is useful for distinguishing unmapped geography, missing measurements, and intentionally missing service-specific fields. It does not execute while commented out.

Cell 47 | Introduce holiday features

MARKDOWN / DOCUMENTATION

CELL 47 / ORIGINAL MARKDOWN

```
1  ## Holidays
```

What Cell 47 does

This Markdown title marks the addition of calendar-based trip context.

Cell 48 | Import the holiday calendar package

CODE / ACTIVE SOURCE

CELL 48 / ORIGINAL CODE

```
1  import holidays
```

What Cell 48 does

import holidays makes the calendar library available, provided it is installed in the current kernel environment. Importing it does not yet attach holiday information to any trip.

Cell 49 | Create holiday-name and holiday-indicator features

CODE / ACTIVE SOURCE

CELL 49 / ORIGINAL CODE

```
# Create collection of New York holidays
ny_holidays = holidays.US(subdiv="NY", years=[2024, 2025, 2026])

pickup_date = taxi_weather_df["pickup_datetime"].dt.date

taxi_weather_holiday_df = taxi_weather_df.copy()

taxi_weather_holiday_df["holiday_name"] = pickup_date.map(ny_holidays)
taxi_weather_holiday_df["is_holiday"] = taxi_weather_holiday_df["holiday_name"].notna().astype(int)
```

Reads: taxi_weather_df and the holidays package
Creates or changes: taxi_weather_holiday_df with holiday_name and is_holiday

What Cell 49 does

holidays.US(subdiv="NY", years=[2024, 2025, 2026]) constructs the requested New York calendar. pickup_datetime.dt.date extracts calendar dates without time-of-day.

The cell copies taxi_weather_df into taxi_weather_holiday_df, maps dates to holiday names, and turns a nonmissing holiday name into integer 1. Other dates receive 0 in is_holiday.

The calendar defines which dates count as holidays; it does not establish that each listed date has the same effect on traffic or demand. The year list is also hardcoded and should be deliberately kept consistent with the study period.

Cell 49: key statements and connections

Lines 8-9. The first expression returns a holiday name or a missing value. notna turns that into True/False, and astype(int) turns the boolean into 1/0. Missing holiday names are expected for ordinary days.

Cell 50 | Preview trips labeled as holidays

CODE / ACTIVE SOURCE

CELL 50 / ORIGINAL CODE

```
taxi_weather_holiday_df[taxi_weather_holiday_df["is_holiday"] == 1].head()
```

What Cell 50 does

The boolean filter selects is_holiday == 1 and head displays five examples. The saved output includes Labor Day trips. This checks that at least some expected dates receive holiday labels; it is not a statistical comparison of holiday and ordinary trips.

Cell 51 | Count sample trips by holiday

CODE / ACTIVE SOURCE

CELL 51 / ORIGINAL CODE

```
1 taxi_weather_holiday_df["holiday_name"].value_counts()
```

What Cell 51 does

value_counts summarizes the frequency of each nonmissing holiday name. Counts depend on sampled trip frequency and how many occurrences of each holiday lie inside the study window. A higher count alone does not show that a holiday increases demand. Normalization and a comparison design would be needed.

Cells 52-73: Legacy pandas workflow

Legacy section. These cells are retained from an older yellow-only workflow. Several use obsolete column names, and Cell 66 actively overwrites df_cleaned. Read them for the concepts; keep them outside the intended execution path until reconciled.

Cell 52 | Warn that the following section is legacy code

MARKDOWN / DOCUMENTATION

CELL 52 / ORIGINAL MARKDOWN

```
1 # Pandas Cleaning and Exploration Please don't delete yet but if you run it there will be
  errors
```

What Cell 52 does

This Markdown cell explicitly says that the pandas cleaning and exploration cells are being retained even though running them can cause errors. They reflect a yellow-only workflow whose names and assumptions no longer match the combined dataset.

Cell 53 | Introduce older sample-coverage checks

MARKDOWN / DOCUMENTATION

CELL 53 / ORIGINAL MARKDOWN

```
1 ## Verifying a good sample of the data was generated
```

What Cell 53 does

This subsection title introduces checks of shape, timestamp range, and counts by calendar period. The checks are conceptually useful, but the following code uses the earlier schema.

Cell 54 | Inspect the current df shape

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 54 / ORIGINAL CODE

```
1 df.shape
```

What Cell 54 does

df.shape returns the pair (number of rows, number of columns). The saved output is (500000, 19), which differs from the newer combined dataset. It is evidence of an earlier dataframe state, not the expected size of the current 5,000-per-file sample.

Cell 55 | Check the earliest pickup timestamp

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 55 / ORIGINAL CODE

```
1 df['tpep_pickup_datetime'].min()
```

What Cell 55 does

min() finds the earliest value in the old tpep_pickup_datetime field. The saved result is January 1, 2009, outside the requested file range. That is a useful clue that some source records contain anomalous timestamps.

Under the current combined schema, the corresponding column is pickup_datetime. The existing expression therefore raises KeyError when df contains the newer schema. Selecting monthly filenames does not enforce a valid timestamp range inside every record.

Cell 56 | Empty cell

EMPTY / NO EFFECT

The original cell is empty.

What Cell 56 does

This cell has no effect.

Cell 57 | Check the latest pickup timestamp

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 57 / ORIGINAL CODE

```
1 df['tpep_pickup_datetime'].max()
```

What Cell 57 does

max() supplies the counterpart to the earliest-date check. The saved result is August 31, 2026. The expression still uses the obsolete tpep_pickup_datetime name, so it needs the unified name to work with the current combined dataframe.

Cell 58 | Extract pickup month and year

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 58 / ORIGINAL CODE

```
1  df['pickup_month'] = df['tpep_pickup_datetime'].dt.month
2  df['pickup_year'] = df['tpep_pickup_datetime'].dt.year
```

What Cell 58 does

The .dt.month and .dt.year accessors derive calendar components from a datetime column and assign them as new columns. These features are useful for coverage checks and potentially modeling. The source column name is from the old yellow-only schema and fails against the current df.

Cell 59 | Count nonmissing distances by year

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 59 / ORIGINAL CODE

```
1  record_counts_by_year = df.groupby('pickup_year')['trip_distance'].count()
2  record_counts_by_year
```

What Cell 59 does

groupby("pickup_year")["trip_distance"].count() groups records by year and counts nonmissing distance entries. count differs from size: size counts rows regardless of missing values in a selected field.

The saved result includes the two 2009 rows and uneven later-year counts. It belongs to the legacy dataframe. The current common distance field is trip_miles.

Cell 60 | Count nonmissing distances by calendar month

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 60 / ORIGINAL CODE

```
1  record_counts = df.groupby('pickup_month')['trip_distance'].count()
```

What Cell 60 does

This groups by month number alone, so August records from different years are pooled. It stores the result in record_counts without displaying it. It still relies on the legacy month feature and trip_distance column.

Cell 61 | Inspect month-and-year coverage together

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 61 / ORIGINAL CODE

```
record_counts_mon_year = df.groupby(['pickup_month', 'pickup_year'])['trip_distance'].count(
)
record_counts_mon_year
```

What Cell 61 does

Grouping by pickup_month and pickup_year distinguishes the same calendar month in different years. The result uses a multi-level index.

This is a better coverage check than pooling months across years. The unusually large saved August 2024 count reflects an older sampling state and should not be interpreted as the outcome of the current equal monthly sampling design.

Cell 62 | Inspect DuckDB database and memory statistics

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 62 / ORIGINAL CODE

```
size_info = con.sql("SELECT * FROM pragma_database_size()").df()
print(size_info)
```

What Cell 62 does

pragma_database_size() reports database-level size, storage, and memory statistics. The saved 0-byte database size is consistent with using an in-memory database without a persistent database file.

It does not mean the raw Parquet files, pandas copies, or overall Python process use no storage or memory. Different layers of the workflow have separate resource use.

Cell 63 | Inspect the legacy dataframe schema

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 63 / ORIGINAL CODE

```
df.info()
```

What Cell 63 does

info() displays the current df column names, nonmissing counts, types, and memory estimate. The saved output has 100,000 rows and yellow-specific names, unlike the saved shape in Cell 54 and the newer combined data. This is another execution-state inconsistency.

Cell 64 | Mark the old pandas cleaning as no longer needed

MARKDOWN / DOCUMENTATION

CELL 64 / ORIGINAL MARKDOWN

```
1  ## Cleaning the  in pandas no longer needed (please don't delete yet)
```

What Cell 64 does

This Markdown note says the older cleaning code is retained for reference. The shared SQL cleaning has superseded much of it, but the source statements below are still executable unless skipped or disabled.

Cell 65 | Preview the legacy yellow-only rows

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 65 / ORIGINAL CODE

```
1  df.head()
```

What Cell 65 does

head() displays five rows using the original yellow schema and previously derived month/year columns. It is a preview only; no cleaning occurs in this cell.

Cell 66 | Replace df_cleaned with whatever df currently contains

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 66 / ORIGINAL CODE

```
1  df_cleaned = df.copy()
2  len(df_cleaned)
```

Reads: Whatever df refers to at execution time
Creates or changes: df_cleaned is replaced, affecting downstream code

What Cell 66 does

df_cleaned = df.copy() is a new assignment, not a validation of the earlier cleaned result. It overwrites the reference established in Cell 29. len then reports the number of rows in the replacement.

Because df has been used for enrichment and is later reused for Spark, this generic variable name creates execution-order sensitivity. Even a successful cell can make later code read a different dataframe than intended.

> Active state mutation inside a legacy section: this can change the input to every later plot and model. Keep the obsolete workflow outside the intended execution path.

Cell 67 | Count suspicious pickup-location codes

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 67 / ORIGINAL CODE

```
df_PULocationID = df.groupby("PULocationID")['passenger_count'].count().reset_index()
df_PULocationID[(df_PULocationID['PULocationID'] == 264) | (df_PULocationID['PULocationID'] == 265)]
```

What Cell 67 does

The groupby counts nonmissing passenger_count values for each PULocationID and resets the index into a regular column. The final filter displays IDs 264 and 265. This diagnoses records; it does not remove them.

The unified name is pickup_location_id. Also, passenger_count is missing for rideshare in the combined schema, so counting that field is not a reliable all-service row count.

Cell 67: key statements and connections

First assignment. groupby produces one group per location code. count counts nonmissing passenger_count entries. reset_index makes the group key a regular dataframe column so it can be filtered by name.

Cell 68 | Count suspicious drop-off-location codes

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 68 / ORIGINAL CODE

```
df_DOlocation = df.groupby("DOLocationID")['passenger_count'].count().reset_index()
df_DOlocation[(df_DOlocation['DOLocationID'] == 264) | (df_DOlocation['DOLocationID'] == 265)]
```

What Cell 68 does

This repeats Cell 67 for DOLocationID. The same limitations apply: it uses an obsolete name and counts nonmissing passenger_count rather than every trip. The current common endpoint name is dropoff_location_id.

Cell 69 | Remove the specified endpoint codes in pandas

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 69 / ORIGINAL CODE

```
# The data we read into using SQL we already removed rows that have passenger_count < 1, total amount < 1 , and trip_distance < 0. Now we want to clean our data some more


# Remove rows with values PULocationID, DOLocationID = 264, 265
# 264 = Unknown
# 265 = N/A / Outside of NYC

df_cleaned = df_cleaned[(df_cleaned['PULocationID'] != 265) & (df_cleaned['DOLocationID'] != 265) & (df_cleaned['PULocationID'] != 264) & (df_cleaned['DOLocationID'] != 264)]
len(df_cleaned)
```

What Cell 69 does

Four boolean conditions retain rows whose pickup and drop-off codes are neither 264 nor 265. The & operator combines elementwise conditions with AND; each comparison must be parenthesized.

This duplicates the shared location filtering in Cell 27, using the older field names. The saved result of 99,743 rows belongs to the earlier yellow-only run. The comment about earlier SQL thresholds is not a precise substitute for inspecting the actual active SQL.

Cell 69: key statements and connections

Boolean selection. df_cleaned[mask] retains rows where the mask is true. All four comparisons must be true. Assigning the result back to df_cleaned replaces the previous dataframe reference.

Cell 70 | Check the legacy cleaned shape

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 70 / ORIGINAL CODE

```
df_cleaned.shape
```

What Cell 70 does

The saved shape (99743, 22) confirms the dimensions of that historical dataframe after location filtering. This is a diagnostic expression with no mutation.

Cell 71 | Keep selected payment categories

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 71 / ORIGINAL CODE

```

# Remove rows with the following payment_types = 3, 4, 5, 6
# 0 = Flex Fare - an upfront, fixed-price fare offered when someone books a NYC yellow or green taxi through a participating e-hail app,
# rather than paying the traditional metered fare. It's similar to how Uber or Lyft provides a price before you confirm a ride.
# Price is determined before the trip
# 3 = No charge
# 4 = Dispute
# 5 = Unknown
# 6 = Voided Trip

df_cleaned = df_cleaned[(df_cleaned['payment_type'] != 0) & (df_cleaned['payment_type'] != 3) & (df_cleaned['payment_type'] != 4) & (df_cleaned['payment_type'] != 5) & (df_cleaned['payment_type'] != 6)].copy()
len(df_cleaned)
```

What Cell 71 does

The boolean filter excludes payment codes 0, 3, 4, 5, and 6. Within the documented set, that leaves 1 (credit card) and 2 (cash). Code 0 denotes a Flex Fare trip [4]. Excluding it narrows the study scope rather than proving those trips are invalid.

This duplicates the current yellow-view payment filtering. The saved legacy count becomes 99,400. Rideshare payment_type is missing by construction, so this yellow-specific logic must not be transferred blindly to a combined service table.

Cell 72 | Inspect the remaining payment distribution

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 72 / ORIGINAL CODE

```
payment_types_df = df_cleaned.groupby("payment_type")['passenger_count'].count()
payment_types_df
```

What Cell 72 does

The groupby counts nonmissing passenger values by payment_type. Saved results are 87,069 credit-card trips and 12,331 cash trips, totaling 99,400. These are historical sample counts, not a newly computed population distribution.

Cell 73 | Drop records missing required yellow-trip fields

CODE / ACTIVE SOURCE / LEGACY SCHEMA

CELL 73 / ORIGINAL CODE

```
#the goal is to remove any row NA / null values for the columns listed below. We believe that these columns are essential for validating the trip actually
# happened and is not a bogus trip.
# tpep_pickup_datetime - use to create new column trip_duration
# tpep_dropoff_datetime  - use to create new column trip_duration
# passenger_count
# trip_distance
# payment_type
# RateCodeID - Pricing Structure Applied
# PULocationID
# DOLocationID
# total_amount - Does not include cash tips, but DOES include credit card tips (tip_amount = 0.0 for all cash payment trips)
df_cleaned = df_cleaned.dropna(subset = ["passenger_count", "tpep_pickup_datetime", "tpep_dropoff_datetime", "trip_distance",
                                         "payment_type", "RatecodeID", "PULocationID", "DOLocationID", "total_amount"  ])
len(df_cleaned)
```

What Cell 73 does

dropna(subset=...) excludes rows missing any listed field: passenger count, pickup/drop-off timestamps, distance, payment type, rate code, endpoint IDs, or total amount. The saved row count does not change in that historical run.

The column list uses the original yellow schema. Renaming the columns alone would not make this suitable for both services: requiring passenger_count or payment_type would exclude rideshare records whose values are deliberately unavailable. Completeness rules must match each field's meaning and service applicability.

Cell 73: key statements and connections

dropna subset. subset limits the completeness requirement to the named columns. It does not require every other field to be present, and it does not validate the magnitude or plausibility of nonmissing values.

Cells 74-80: Exploratory visualizations

Cell 74 | Introduce exploratory plots

MARKDOWN / DOCUMENTATION

CELL 74 / ORIGINAL MARKDOWN

```
# Visualizations and Describing our Data
```

What Cell 74 does

This Markdown title starts the visual exploration section.

Cell 75 | Build a numeric correlation heatmap

CODE / ACTIVE SOURCE

CELL 75 / ORIGINAL CODE

```
corr_mat = df_cleaned.corr(numeric_only=True)
plt.figure(figsize=(20,20))
sns.heatmap(corr_mat, cmap="RdBu", annot=True, vmin = -1, vmax=1, center =0)
plt.title("Yellow Taxi Data Correlation Matrix")

plt.show()
```

Reads: Numeric columns of df_cleaned
Creates or changes: corr_mat and a displayed heatmap

What Cell 75 does

corr(numeric_only=True) calculates pairwise Pearson correlations among numeric columns. The heatmap fixes the range at -1 to 1, centers the colors at zero, and prints each coefficient. Values near +1 or -1 indicate strong positive or negative linear association; values near zero indicate little linear association.

Numeric IDs are not quantitative measurements: a larger zone ID does not mean farther north or farther away. Pairwise missing-value handling can also mean different cells in the matrix use different trip subsets. Fare components are mechanically related to total charges.

The title says Yellow Taxi, although df_cleaned may contain both services. This chart requires careful labels and feature selection before it can support a clear conclusion.

Cell 75: key statements and connections

Lines 2-6. figure creates the drawing surface; heatmap draws the correlation matrix; title labels it; show displays it. Those steps do not change the trip dataframe.

Cell 76 | Preserve an inactive duplicate heatmap

COMMENTED CODE / INACTIVE

CELL 76 / ORIGINAL CODE

```
# Correlation Plot
# corr_mat = df_cleaned.corr(numeric_only=True)
# plt.figure(figsize=(20,20))
# sns.heatmap(corr_mat, cmap="RdBu", annot=True, vmin = -1, vmax=1, center =0)
# plt.title("Yellow Taxi Data Correlation Matrix")

# plt.show()
```

What Cell 76 does

All lines repeat the prior plot as comments. The cell currently does nothing. A visible plot underneath is a saved output rather than evidence that the commented source executed.

Cell 77 | Plot recorded passenger counts

CODE / ACTIVE SOURCE

CELL 77 / ORIGINAL CODE

```
df_cleaned['passenger_count'].hist(bins=15, color='blue')
plt.title("Rides by Total Passenger Count")
plt.ylabel("Number of Rides")
plt.xlabel("Number of Passengers")
```

What Cell 77 does

The histogram counts records in bins of passenger_count and adds a title and axis labels. Missing passenger counts are not plotted. With the combined schema, this therefore largely describes yellow trips rather than both services.

Passenger count is discrete. Integer-aligned bins or a bar chart of value counts would make the categories easier to read than a generic 15-bin histogram.

Cell 78 | Inspect the dataframe before further plots

CODE / ACTIVE SOURCE

CELL 78 / ORIGINAL CODE

```
df_cleaned.info()
```

What Cell 78 does

The saved info output shows the combined data and a pickup_hour column. That column is created in Cell 80, after this cell in file order. This is further evidence that cells were executed out of order or that outputs were retained from another state.

Cell 79 | Plot distance against total charge

CODE / ACTIVE SOURCE

CELL 79 / ORIGINAL CODE

```
#Correlation plot between trip distance and total trip amount
sample_df = df_cleaned.sample(n=min(100_000, len(df_cleaned)), random_state=42)

plt.figure(figsize=(9, 6))

sns.scatterplot(
    sample_df,
    x="trip_miles",
    y="total_charged",
    alpha=0.3,
    s=15
)

plt.title("Trip Distance vs. Total Trip Amount")
plt.xlabel("Trip Distance (miles)")
plt.ylabel("Total Amount Charged ($)")
plt.xlim(0, sample_df["trip_miles"].quantile(0.99))
plt.ylim(
    sample_df["total_charged"].quantile(0.01),
    sample_df["total_charged"].quantile(0.99)
)

plt.show()
```

Reads: df_cleaned
Creates or changes: sample_df and a distance-versus-charge scatter plot

What Cell 79 does

sample(n=min(100000, len(df_cleaned)), random_state=42) chooses at most 100,000 trips for plotting and avoids requesting more rows than exist. Seaborn plots trip_miles on the horizontal axis and total_charged on the vertical axis. Small, partly transparent markers reduce overlap.

The x-axis ends at the sample's 99th distance percentile, and the y-axis covers the 1st through 99th charge percentiles. These limits hide extreme points from view; they do not remove those records from the dataframe or the model.

The plot pools services because it does not use service_type for color or panels. A pooled relationship can conceal differences in service composition, fees, and trip geography.

Cell 79: key statements and connections

Lines 6-12. Each marker is one sampled trip. alpha controls transparency and s controls marker size. Neither parameter changes the fitted data or computes a correlation coefficient.

Lines 17-21. quantile returns cutoff values from sample_df. xlim and ylim set viewing limits only; the hidden extreme rows still exist.

Cell 80 | Aggregate trip frequency and charges by pickup hour

CODE / ACTIVE SOURCE

CELL 80 / ORIGINAL CODE

```
#Pickup times and drop off times
df_cleaned["pickup_datetime"] = pd.to_datetime(
    df_cleaned["pickup_datetime"],
    errors="coerce"
)

df_cleaned["pickup_hour"] = df_cleaned["pickup_datetime"].dt.hour

hourly = (
    df_cleaned.groupby("pickup_hour")
      .agg(
          trip_count=("total_amount", "size"),
          avg_total_amount=("total_amount", "mean"),
          median_total_amount=("total_amount", "median")
      )
      .reset_index()
)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.barplot(data=hourly, x="pickup_hour", y="trip_count", ax=axes[0])
axes[0].set_title("Number of Trips by Pickup Hour")
axes[0].set_xlabel("Hour of Day")

sns.lineplot(
    data=hourly,
    x="pickup_hour",
    y="median_total_amount",
    marker="o",
    ax=axes[1]
)
axes[1].set_title("Median Trip Amount by Pickup Hour")
axes[1].set_xlabel("Hour of Day")
axes[1].set_ylabel("Median Amount ($)")

plt.tight_layout()
plt.show()
```

Reads: df_cleaned pickup time and charge column
Creates or changes: pickup_hour; intended hourly table and two plots; aggregation currently fails

What Cell 80 does

The cell parses pickup_datetime with errors="coerce", which converts invalid values to NaT, then extracts hours 0 through 23. A groupby is intended to calculate trip count, average charge, and median charge for each pickup hour.

The two plots would display hourly sample trip counts and median charges. The mean is calculated but not plotted. Median is less sensitive to very large charges than mean.

The saved KeyError occurs because all three aggregations reference total_amount, but the combined field is total_charged. pickup_hour is assigned before the failure, so that column can remain in memory even when the rest of the cell fails.

Cell 80: key statements and connections

Lines 9-17. Named aggregation uses output_name=(input_column, operation). reset_index turns the grouping hour back into a normal column for plotting.

Lines 19-34. subplots creates two axes. ax=axes[0] directs the bars to the first panel and ax=axes[1] directs the line to the second. The cell fails before creating these plots in the saved run.

> The source above is unchanged: the three named aggregations still use total_amount. The unified dataframe supplies total_charged. Changing those three input-column references is the immediate correction; it has not been applied to the original notebook.

Cells 81-102: Spark and model draft

> **Model status.** No successful trained model or trustworthy performance result is established by this draft. The walkthrough explains intended behavior and identifies the missing prerequisites.

Cell 81 | Raise the question of using Spark

MARKDOWN / DOCUMENTATION

CELL 81 / ORIGINAL MARKDOWN

```
# Do we need spark?
```

What Cell 81 does

This Markdown title asks whether Spark is needed. The roughly 250,000-row working sample can reasonably be handled by DuckDB, pandas, and scikit-learn. The notebook does not demonstrate a workload that requires distributed execution.

Cell 82 | Create a Spark session and read an export

CODE / ACTIVE SOURCE

CELL 82 / ORIGINAL CODE

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("TaxiFarePrediction") \
    .getOrCreate()

df = spark.read.parquet("taxi_ml_data.parquet")
```

Reads: Spark installation and taxi_ml_data.parquet
Creates or changes: spark session; df reassigned to a Spark DataFrame

What Cell 82 does

SparkSession.builder.appName(...).getOrCreate() creates or reuses a Spark session. spark.read.parquet then expects taxi_ml_data.parquet to exist.

The earlier export is commented out, so this notebook does not establish that the required file exists. The cell also reassigns df to a Spark DataFrame, whereas earlier df was a pandas DataFrame. Those objects have different APIs.

The subsequent scikit-learn cells still use df_cleaned. Starting Spark here does not make those models distributed or connect Spark to the training workflow.

Cell 83 | Introduce machine learning

MARKDOWN / DOCUMENTATION

CELL 83 / ORIGINAL MARKDOWN

```
# Machine Learning
```

What Cell 83 does

This Markdown title marks the model-building portion, which remains a draft.

Cell 84 | Introduce the model imports

MARKDOWN / DOCUMENTATION

CELL 84 / ORIGINAL MARKDOWN

```
## Importing Machine Learning Packages
```

What Cell 84 does

This Markdown subtitle precedes the scikit-learn imports.

Cell 85 | Import preprocessing, model, and metric classes

CODE / ACTIVE SOURCE

CELL 85 / ORIGINAL CODE

```
# importing machine learning packages
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import (mean_squared_error, r2_score, classification_report)
```

What Cell 85 does

StandardScaler standardizes numeric features; OneHotEncoder converts categories into indicator columns; PCA creates lower-dimensional numeric components. train_test_split creates training and test subsets. LinearRegression predicts numeric outcomes.

ColumnTransformer can apply different preprocessing to different columns, and Pipeline can connect transformations to a model. mean_squared_error and r2_score are regression metrics. LogisticRegression and classification_report are for classification and are unused in the shown fare-regression path.

Importing these classes does not execute preprocessing. In particular, the encoder and pipeline tools are imported but never assembled into the missing preprocessing workflow.

Cell 86 | Leave a placeholder for one-hot encoding

MARKDOWN / DOCUMENTATION

CELL 86 / ORIGINAL MARKDOWN

```
## One Hot Encoding
```

What Cell 86 does

This is only a Markdown heading. No encoding occurs before the later scaler and model calls.

One-hot encoding would represent categories using indicator columns. For example, a service-type category can be represented numerically so a linear model can learn an adjustment for rideshare versus yellow taxi. The same conceptual issue applies to zone IDs and flags. Raw strings such as N cannot be passed directly to the numeric scaling step.

Cell 87 | Introduce principal component analysis

MARKDOWN / DOCUMENTATION

CELL 87 / ORIGINAL MARKDOWN

```
## PCA
```

What Cell 87 does

This Markdown heading introduces PCA. PCA is a feature transformation, not a fare-prediction model. It learns directions of variation in the inputs without using the target charge.

Cell 88 | Inspect the dataframe selected for modeling

CODE / ACTIVE SOURCE

CELL 88 / ORIGINAL CODE

```
df_cleaned.info()
```

What Cell 88 does

The saved info result contains 249,550 rows and 37 columns, differing from earlier saved row counts. It shows the common trip schema rather than the full weather, geography-name, and holiday feature set.

This supports two separate observations: the outputs come from multiple states, and the model path uses the earlier dataframe rather than the enriched result.

Cell 89 | Preview model-source records

CODE / ACTIVE SOURCE

CELL 89 / ORIGINAL CODE

```
1 df_cleaned.head()
```

What Cell 89 does

head() shows yellow trips with missing rideshare-only fields. Those NULLs are a consequence of the schema alignment. They need a service-aware modeling decision, not an assumption that each missing field reflects a corrupt record.

Cell 90 | Define model features and the prediction target

CODE / ACTIVE SOURCE

CELL 90 / ORIGINAL CODE

```
1 # Features dataframe into variable called "features"
2 features = df_cleaned.drop(columns = ["service_type", "total_charged", "pickup_datetime",
  "dropoff_datetime", 'request_datetime','on_scene_datetime'])
3
4 # TO-DO: store the classification target variable into "target"
5 target = df_cleaned['total_charged']
6 target
```

Reads: df_cleaned, not the enriched dataframe
Creates or changes: features and target

What Cell 90 does

features is created by dropping service_type, total_charged, and four timestamp columns from df_cleaned. target is df_cleaned["total_charged"]. The intended task is therefore numeric regression: predict a trip's charge from the retained attributes. The comment calling the target a classification variable is incorrect.

Several strings and structural NULLs remain in features. The explicit service_type field is removed, although other service-specific fields can still indirectly reveal the service. The weather-and-holiday table is not used.

The retained features include fare_amount, base_fare, taxes, tolls, and other charge components. For a prediction made before a ride, these can give away the answer or contain information unavailable at prediction time. That is target leakage or an information-availability mismatch.

The team must first define the prediction moment. Actual completed distance, duration, final driver pay, and final bill components are not automatically valid inputs for a pre-trip quote. Explanatory analysis after a ride and advance prediction require different feature choices.

Cell 90: key statements and connections

Line 2. drop(columns=...) creates a new feature table without the listed columns. It does not mutate df_cleaned here because the result is assigned separately and inplace is not requested.

Lines 5-6. target is a one-dimensional Series. The final bare target expression asks the notebook to display it; it does not train a model.

> A high score from bill components would not demonstrate a useful advance fare predictor. Define the target and permitted inputs before interpreting any model accuracy.

Cell 91 | Create the 80/20 training and test split

CODE / ACTIVE SOURCE

CELL 91 / ORIGINAL CODE

```
1  # Assign appropriate value to seed and conduct 80/20 train-test split with random_state =
   seed
2  seed = 42
3  X_train, X_test, y_train, y_test = train_test_split(features,target, test_size = 0.2,
   random_state = seed)
```

Reads: features and target
Creates or changes: X_train, X_test, y_train, y_test

What Cell 91 does

train_test_split allocates approximately 80% of rows to fitting and 20% to held-out evaluation, using random_state=42 for a repeatable split of the same input. X_train and X_test are feature tables; y_train and y_test are the corresponding charge values.

A random split measures performance on held-out trips from the same mixed study period. If the intended use is future-trip prediction, a chronological holdout is a closer evaluation of that setting. Preprocessing parameters must be learned from training data only [9].

Cell 91: key statements and connections

Line 3. Passing features and target together preserves row alignment in both returned subsets. All four returned objects correspond to the same random allocation of trips.

Cell 92 | Retain an inactive numeric-only selection

COMMENTED CODE / INACTIVE

CELL 92 / ORIGINAL CODE

```
1  #Get the numeric columns
2  # numeric_cols = df_cleaned.select_dtypes(include=["int64", "float64", "int32"]).columns
3  # X = df_cleaned[numeric_cols]
```

What Cell 92 does

The commented select_dtypes expression would select numeric columns from df_cleaned. That can remove strings but does not resolve missing values, numeric category IDs, or prediction-time availability.

Selecting from the full dataframe could also include total_charged itself. The target must be excluded explicitly. This cell is not an implemented fix.

Cell 93 | Retain an inactive whole-dataset scaling attempt

COMMENTED CODE / INACTIVE

CELL 93 / ORIGINAL CODE

```
# Standardize the data
# scaler = StandardScaler()
# X_scaled = scaler.fit_transform(X)
```

What Cell 93 does

The commented scaler would fit to X as a whole. If done before a train/test split, it would learn preprocessing parameters using the test set. The later fit-on-training, transform-on-test pattern is the appropriate separation [9]. This cell has no current effect.

Cell 94 | Standardize inputs and attempt PCA

CODE / ACTIVE SOURCE

CELL 94 / ORIGINAL CODE

```
# TO-DO: Intermediate step to address fact that PCA is not scale-invariant
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# TO-DO: Instantiate and Fit PCA
pca = PCA(n_components=10, random_state=42)
pca.fit(X_train_scaled)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

Reads: Train/test feature matrices
Creates or changes: Intended scaler, scaled matrices, PCA object and PCA matrices; current scaling fails

What Cell 94 does

StandardScaler.fit_transform(X_train) learns each numeric feature's training mean and standard deviation, then rescales it. scaler.transform(X_test) applies those same parameters to held-out rows. Conceptually, z = (x - training mean) / training standard deviation.

The saved execution fails during scaling with ValueError: could not convert string to float: N. Encoding was never implemented. That failure prevents the later PCA statements from completing in that run. Unresolved missing values would also need handling before this PCA and regression route can work.

PCA(n_components=10) would create ten weighted combinations of numeric inputs, chosen to retain variation in the input space. The code calls pca.fit and then pca.fit_transform on the same training data; fit_transform already fits, so the first call is redundant [11].

Cell 94: key statements and connections

Lines 2-4. fit learns preprocessing parameters. transform applies learned parameters. The test data must use the training parameters, not its own separately fitted scaler.

Lines 8-11. PCA is another fitted transformation. The second training fit is redundant. X_train_pca and X_test_pca are new matrices, but neither is consumed by the later regression cells.

The source above includes both pca.fit and pca.fit_transform. The first fit is redundant. Encoding and appropriate missing-value handling are prerequisites, so removing the duplicate fit alone will not make this modeling cell work.

Cell 95 | Read individual and cumulative explained variance

CODE / ACTIVE SOURCE

CELL 95 / ORIGINAL CODE

```
1  # TO-DO: Save the explained variance ratios into variable called "explained_variance_ratios"
2  explained_variance_ratios = pca.explained_variance_ratio_
3  explained_variance_ratios
4
5  # TO-DO: Save the CUMULATIVE explained variance ratios into variable called "cum_evr"
6  cum_evr = np.cumsum(pca.explained_variance_ratio_)
7  cum_evr
```

Reads: A successfully fitted pca
Creates or changes: explained_variance_ratios and cum_evr

What Cell 95 does

explained_variance_ratio_ reports the fraction of input variance represented by each retained principal component. np.cumsum accumulates those fractions. For an illustrative sequence 0.40, 0.25, 0.15, the cumulative values are 0.40, 0.65, 0.80.

This describes input variation, not the fraction of fare variation the eventual regression explains. The cell depends on a successfully fitted PCA object, so it cannot establish a new valid result after the preceding failure.

Cell 95: key statements and connections

Lines 2 and 6. The trailing underscore in explained_variance_ratio_ marks a learned fitted attribute in this scikit-learn API. cumsum converts individual fractions into a running total.

Lines 3 and 7. In ordinary notebook display behavior, the last bare expression is the one automatically shown. To display both intermediate and cumulative arrays reliably, use explicit display calls.

Cell 96 | Plot the cumulative variance curve

CODE / ACTIVE SOURCE

CELL 96 / ORIGINAL CODE

```
1  # TO-DO: find optimal num components to use (n) by plotting explained variance ratio (2
   points)
2  components = np.arange(1, len(explained_variance_ratios) + 1)
3  plt.figure(figsize=(10,6))
4  plt.plot(components, cum_evr, marker = "o" , linestyle = "-")
5  plt.axhline(y=0.80, color="darkred", linestyle="--", alpha = 0.8, label = "80% variance")
6  plt.xlabel("Number of Components")
7  plt.ylabel("Cumulative Explained Variance Ratio")
8  plt.xticks(components)
9  plt.legend(loc="lower right")
10 plt.title("Cumulative Explained Variance Ratio Plot")
```

CELL 96 / ORIGINAL CODE / CONTINUED

```
11 plt.show()
```

Reads: explained_variance_ratios and cum_evr
Creates or changes: components and a cumulative-variance plot

What Cell 96 does

The plot uses component counts on the horizontal axis and cumulative explained variance on the vertical axis. The 0.80 horizontal line is a reference threshold.

Because PCA was limited to ten components, the chart only covers those ten. It cannot show how many additional components would be needed if the curve has not reached the threshold. An 80% input-variance threshold does not establish the best predictive model; that needs validation performance.

Cell 96: key statements and connections

Lines 2, 4, and 5. arange creates component numbers starting at 1. plot draws the cumulative curve. axhline draws a reference threshold; it does not select or refit a new number of components.

Cell 97 | Introduce unscaled linear regression

MARKDOWN / DOCUMENTATION

CELL 97 / ORIGINAL MARKDOWN

```
1 ## Linear Regression (non-scaled)
```

What Cell 97 does

This Markdown subtitle introduces an ordinary least-squares model fitted to the current original features. It does not connect the model to PCA.

Cell 98 | Fit linear regression and score held-out trips

CODE / ACTIVE SOURCE

CELL 98 / ORIGINAL CODE

```
1 # Initialize model with default parameters and fit it on the training set
2 reg = LinearRegression()
3 reg.fit(X_train, y_train)
4
5 # Use the model to predict on the test set and save these predictions as `y_pred`
6 y_pred = reg.predict(X_test)
7
8 # Find the R-squared score and store the value in `lin_reg_score`
9 lin_reg_score = reg.score(X_test, y_test)
```

Reads: X_train, X_test, y_train, y_test
Creates or changes: Intended reg model, test y_pred, and lin_reg_score

What Cell 98 does

LinearRegression.fit learns an intercept and feature coefficients to minimize squared training errors. predict(X_test) generates held-out charge estimates. score(X_test, y_test) returns test R-squared.

R-squared equals 1 for perfect predictions, 0 for the constant predictor equal to the test target mean, and can be negative when predictions perform worse than that baseline. The notebook stores lin_reg_score but does not show a successful resulting score.

The code passes the unprocessed X_train and X_test, which still contain strings and missing values. It also uses neither X_train_pca nor X_test_pca. PCA is currently disconnected from the fitted regression.

Cell 98: key statements and connections

Lines 2-9. The estimator object stores fitted coefficients after fit. predict generates estimates; score evaluates the fitted model. Assigning lin_reg_score stores the number but does not automatically display it.

Cell 99 | Introduce Ridge regression

MARKDOWN / DOCUMENTATION

CELL 99 / ORIGINAL MARKDOWN

```
## Ridge Regression
```

What Cell 99 does

This Markdown subtitle introduces a regularized linear model. Ridge adds a penalty on coefficient magnitudes to the ordinary squared-error objective.

Cell 100 | Standardize inputs for Ridge

CODE / ACTIVE SOURCE

CELL 100 / ORIGINAL CODE

```
# TO-DO: Intermediate step to address scale-invariance
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Reads: X_train and X_test
Creates or changes: Intended new scaler and scaled matrices

What Cell 100 does

This repeats the train-only scaling pattern. Feature scale matters for Ridge because a coefficient penalty interacts with the units of its feature. For example, the coefficient value changes if the same distance is measured in miles versus feet.

The step still encounters the unencoded strings. It also repeats work already attempted earlier, rather than placing the transformations and model in one pipeline.

Cell 101 | Fit Ridge and mix training predictions with a test score

CODE / ACTIVE SOURCE

CELL 101 / ORIGINAL CODE

```
1  # TO-DO: Import required libraries
2  from sklearn.linear_model import Ridge
3  from sklearn.model_selection import train_test_split
4
5  # TO-DO: Initialize model with alpha = 10 (keep other hyperparameters as default values) and
   fit it on the training set
6  reg_ridge = Ridge(alpha = 10)
7
8  # TO-DO: Use the model to predict on the test set and save these predictions as `y_pred
9  reg_ridge.fit(X_train_scaled, y_train)
10 y_pred = reg_ridge.predict(X_train_scaled)
11
12 # TO-DO: Find the R-squared score and store the value in `ridge_score`
13 ridge_score = reg_ridge.score(X_test_scaled, y_test)
```

Reads: Scaled feature matrices and targets
Creates or changes: Intended reg_ridge, training y_pred, and test ridge_score

What Cell 101 does

Ridge(alpha=10) minimizes squared errors plus an L2 penalty proportional to the sum of squared coefficients. alpha controls the penalty strength; the notebook does not show a validation search establishing that 10 is appropriate [12].

The model fits to X_train_scaled and y_train. However, y_pred is generated from X_train_scaled, so it contains training predictions. ridge_score is separately computed using X_test_scaled and y_test, so that score uses held-out data.

For test predictions comparable with y_test, use predict(X_test_scaled). Reusing y_pred also overwrites the linear-regression predictions. Distinct names would make model comparisons clearer. Like the ordinary regression, Ridge does not use the PCA features.

Cell 101: key statements and connections

Lines 8-10. The comment says test set, but the actual predict call passes X_train_scaled. Python follows the code rather than the comment, so y_pred contains training predictions.

Line 13. The separate score call passes test data correctly. Its R-squared can be a test score even though the nearby y_pred variable contains predictions for a different subset.

> The original predict call uses X_train_scaled. For held-out predictions, the input should be X_test_scaled. The score call already uses test data. This explanation identifies the change without silently altering the original code block.

Cell 102 | Empty final cell

EMPTY / NO EFFECT

The original cell is empty.

What Cell 102 does

This cell contains no code or output-producing operation.

Problems to resolve before trusting results

The priorities below distinguish execution blockers from issues that can produce plausible-looking but misleading results. A cell that runs successfully is not automatically answering the intended question.

Priority	Where	Issue and required decision
1 / Run	6	Colab-only Drive import fails locally. Choose the execution environment and define valid paths.
1 / Run	45, 49	Holiday-enriched variable is read before creation. Correct the order or use the already-existing weather dataframe.
1 / Run	52-73	Legacy names conflict with the unified schema. Exclude this workflow from the intended run; prevent Cell 66 from resetting df_cleaned.
1 / Run	80	Hourly aggregation uses total_amount after it was renamed to total_charged.
1 / Run	31, 82	Spark expects an export that the notebook does not create. Remove the disconnected step or implement the intended handoff.
1 / Run	86, 90, 94	Encoding is absent. Numeric preprocessing encounters strings, followed by unresolved missingness.
2 / Meaning	23	Total charges differ in tip treatment and omitted fee components. Define a comparable target before analysis.
2 / Meaning	30, 49, 90	Weather and holiday features never reach the model. Select the intended enriched table explicitly.
2 / Meaning	90	Bill components and post-trip information can give away the target. Define prediction time and permitted inputs.
2 / Meaning	11-16, 25	Equal monthly/service samples do not preserve population proportions. Label counts and decide whether weighting is needed.
2 / Meaning	27, 39	Validate duration, date range, weather-key uniqueness, time alignment, and join row retention.
3 / Evaluate	91, 98, 101	Choose a relevant holdout. Keep test predictions distinct from training predictions and report dollar-scale errors.
3 / Evaluate	94-101	PCA outputs are unused by the regressions. Decide whether to compare with/without PCA or omit that branch.

A coherent modeling workflow

A reliable next version should make one chain of dependencies explicit. This is a design recommendation based on the inspected notebook, not an implementation claim.

1. State the prediction question

For example: estimate a comparable pre-tip trip charge using information available when a ride is requested. Decide whether distance means an available route estimate or an actual completed distance; those support different uses.

2. Define the target consistently

Choose which fees and tips belong in the outcome. Reconcile source fields by service and year. Treat missing charge components deliberately instead of silently assuming every missing component is zero.

3. Select the enriched data and eligible features

Use one clearly named modeling dataframe. Keep meaningful service and geographic categories. Remove target components and fields unavailable at the chosen prediction moment. Make validity and missingness rules explicit.

4. Create the evaluation split

Use a time-based holdout for a future-trip claim, or explain the purpose of a random trip holdout. Do this before fitting learned preprocessing. Preserve a final test set for a one-time performance estimate.

5. Build preprocessing inside a pipeline

Use a ColumnTransformer to apply appropriate numeric handling and categorical encoding. Fit imputation, scaling, and optional dimensionality reduction only on training data. A Pipeline keeps those learned transformations aligned with the model [9, 10].

6. Compare a baseline and candidate models

Start with a simple baseline, then compare LinearRegression and Ridge with defensible feature choices. Tune Ridge on training/validation folds. If PCA is included, actually feed its output to the model and compare against the same model without PCA.

7. Evaluate and interpret

Report MAE or RMSE in dollars alongside R-squared. Compare errors separately by service and relevant groups such as borough, distance range, and time period. Check whether a good pooled score masks a weak subgroup.

8. Reproduce the final result

Restart the kernel and run only the intended workflow in order. Record data period, schema assumptions, package versions, seeds, retained counts, feature lists, and model settings. Save the final cleaned or modeling table if that is part of the agreed workflow.

Operations used throughout the code

Concept	Meaning in this project
Parquet	A column-oriented file format used for full monthly trip records and smaller sample files.
SQL view	A named query definition. Creating it is different from storing a complete materialized result.
Materialization	Actually computing and storing query results, for example in pandas or an exported file.
CTE / WITH	A named subquery used to express the eligible trip set before sampling.
UNION ALL	Append rows from compatible SELECT lists. It preserves duplicates and does not pair equivalent journeys.
Join / merge	Match rows by keys. The weather merge is many trips to one time interval if weather keys are unique.
map	Translate a value through a lookup. Used for zones, boroughs, weather descriptions, and holidays.
Structural NULL	A value missing because the source does not supply that field, such as rideshare passenger count here.
count versus size	count ignores missing entries in a selected column; size counts rows in the group.
Feature / target	A feature is a model input. The target is the numeric trip charge the model tries to predict.
One-hot encoding	Represent categories as indicator columns so a numeric model does not treat labels as measured quantities.
Standardization	Rescale numeric features using training means and standard deviations.
PCA	Construct new input directions that retain variance; it does not optimize fare prediction directly.
Ridge	Linear regression with a coefficient-size penalty controlled by alpha.
Leakage	Giving the training or prediction process information it should not have, such as test-derived preprocessing or final bill components for an advance quote.
MAE / RMSE	Prediction error measures expressed in target units. For trip charges, those units are dollars.

Sources and review scope

The notebook is the primary source for all cell descriptions, saved errors, and historical row counts. Official documentation supports selected explanations of library behavior and dataset semantics. Links were consulted during the review on October 22, 2026. No full dataset execution, new fitted model, or new empirical performance result is claimed.

[1] Primary source: CIS5450_GroupProject_BeginningFile.ipynb
User-provided notebook, reviewed October 22, 2026. The analysis is based on source and saved outputs, not a fresh full execution.

[2] DuckDB: Samples
Reservoir sampling and reproducibility considerations.
https://duckdb.org/docs/stable/sql/samples.html

[3] DuckDB: Parquet Tips
Combining files with different or missing columns using union_by_name.
https://duckdb.org/docs/stable/data/parquet/tips.html

[4] NYC TLC: Yellow Taxi Data Dictionary
Field meanings, payment codes, total amount, tips, and congestion fee.
https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_yellow.pdf

[5] NYC TLC: High Volume FHV Data Dictionary
Rideshare fare components, timestamps, and service-specific fields.
https://www.nyc.gov/assets/tlc/downloads/pdf/data_dictionary_trip_records_hvfhs.pdf

[6] NYC TLC: Trip Record Data
Dataset resources and the 2025 schema addition for cbd_congestion_fee.
https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[7] Open-Meteo: Historical Forecast API
Archived model data and available weather variables.
https://open-meteo.com/en/docs/historical-forecast-api

[8] Open-Meteo: Weather Forecast API documentation
Parameter units and timestamp conventions.
https://open-meteo.com/en/docs

[9] scikit-learn: Common pitfalls
Train/test separation and preprocessing leakage.
https://scikit-learn.org/stable/common_pitfalls.html

[10] scikit-learn: Column Transformer with Mixed Types
Separate preprocessing for numeric and categorical inputs.
https://scikit-learn.org/stable/auto_examples/compose/plot_column_transformer_mixed_types.html

[11] scikit-learn: PCA
Principal components and explained variance.
https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

[12] scikit-learn: Ridge
L2-regularized linear regression and alpha.
https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html